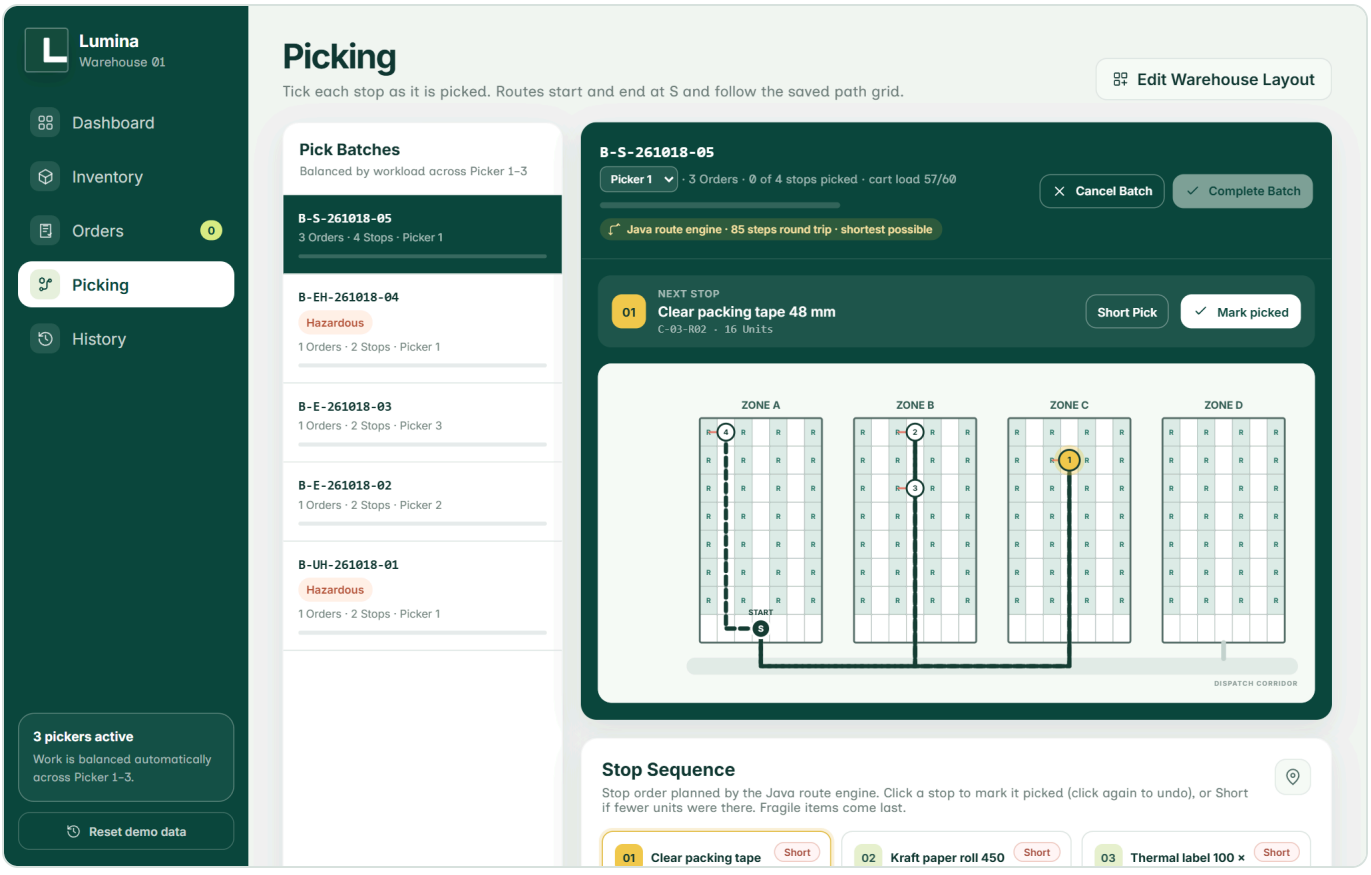
WAREHOUSE PICKING APP

Lumina

Complete code guide: every file and every function explained, from the screens to the Java route engine



The Picking page. The route on the map was planned by Lumina's Java route engine: 85 steps for the round trip, the shortest possible.

What it covers
main.cjs, preload.cjs, route-engine.cjs, renderer.js (160 functions), index.html, styles.css and the 8 Java files

Version
Lumina app 1.7.0 (package "wayline-warehouse-desktop") · Java route engine 1.0

Prepared
October 18, 2026, for presenting Lumina in class. Line numbers refer to the lumina-main folder.

Contents

Part 1 · Introduction to Lumina 4
- What Lumina is 4
- A tour of the five pages 4
- Technologies used 9
- The files in the project 10
- How to run Lumina 10
- Key numbers and rules at a glance 11

Part 2 · How the pieces fit together 12
- The big picture 12
- Why three programs? 12
- The data: one `state` object 13
- Location codes and grid squares 13
- Business rules in the code 14
- The life cycle of an order 14
- How a picking route is planned 14
- Saving and upgrading data 15

Part 3 · The desktop shell: Electron files 16
- main.cjs: the main process 16
- preload.cjs: the safe bridge 17
- route-engine.cjs: talking to Java 17
- index.html: the page frame 19
- styles.css: the look 19

Part 4A · renderer.js: setup, data and rules 21
- Setup and icons 21
- The warehouse model and data upgrades 22
- Stock, cart and batching rules 23
- The saved state 26
- Shared definitions and status rules 28
- Navigation and table filters 29

Part 4B · renderer.js: Dashboard, Inventory and Orders 31
- Dashboard 31
- Inventory 34
- Stock availability and held orders 39
- Orders 40

Part 4C · renderer.js: batching and picking 44
- Turning orders into batches 44
- The Picking page 45

Part 4D · renderer.js: paths, routes, the map and the layout editor 48
- Walking on the grid: breadth-first search 48
- The built-in routing 50
- Talking to the Java engine (desktop app) 51

Drawing the warehouse map 53
The warehouse grid editor 54
Activity history and demo reset 55
Starting the app (the last lines) 56

Part 5 · The Java route engine 57
What the engine does 57
A real request and answer 57
EngineServer.java: the program's entry point 58
Json.java: reading and writing JSON 59
Warehouse.java: the request as Java records 60
WarehouseGraph.java: where a picker can walk 62
ShortestPaths.java: Dijkstra's algorithm 63
RouteOptimizer.java: choosing the best route 64
RoutePlan.java: the answer 67
EngineTest.java: how we know it is correct 67

Part 6 · Walkthrough: the life of one order 71
1. Starting the app 71
2. Creating an order 71
3. Batching 71
4. Planning the best route with Java 71
5. Picking 72
6. Completing, packing and shipping 72

Part 7 · Presenting Lumina in class 74
A suggested slide plan (10–15 minutes) 74
A live demo script 74
Questions you may be asked 75
Limits and ideas for improvement 76
Glossary 76

Appendix · Index of every function 77

Part 1 · Introduction to Lumina

What Lumina is

Lumina is a desktop application for running a small warehouse. It keeps track of the products on the shelves, takes customer orders, groups those orders into picking batches for three warehouse workers ("pickers"), and plans the shortest walking route through the warehouse for each batch. When a picker finishes a batch, Lumina updates the stock and moves the orders on to packing and shipping.

The application has two parts that work together:

- **The Lumina app** (folder `Lumina App (source)`): the window you see. It is an Electron program written in HTML, CSS and plain JavaScript. Almost all of the logic lives in one file, `renderer.js`.
- **The Lumina route engine** (folder `Lumina Java Engine (source)`): a small Java program that the app starts in the background. Its only job is to plan the shortest route for a picking batch. If Java is not available, the app plans routes itself with simpler built-in routing, so it always works.

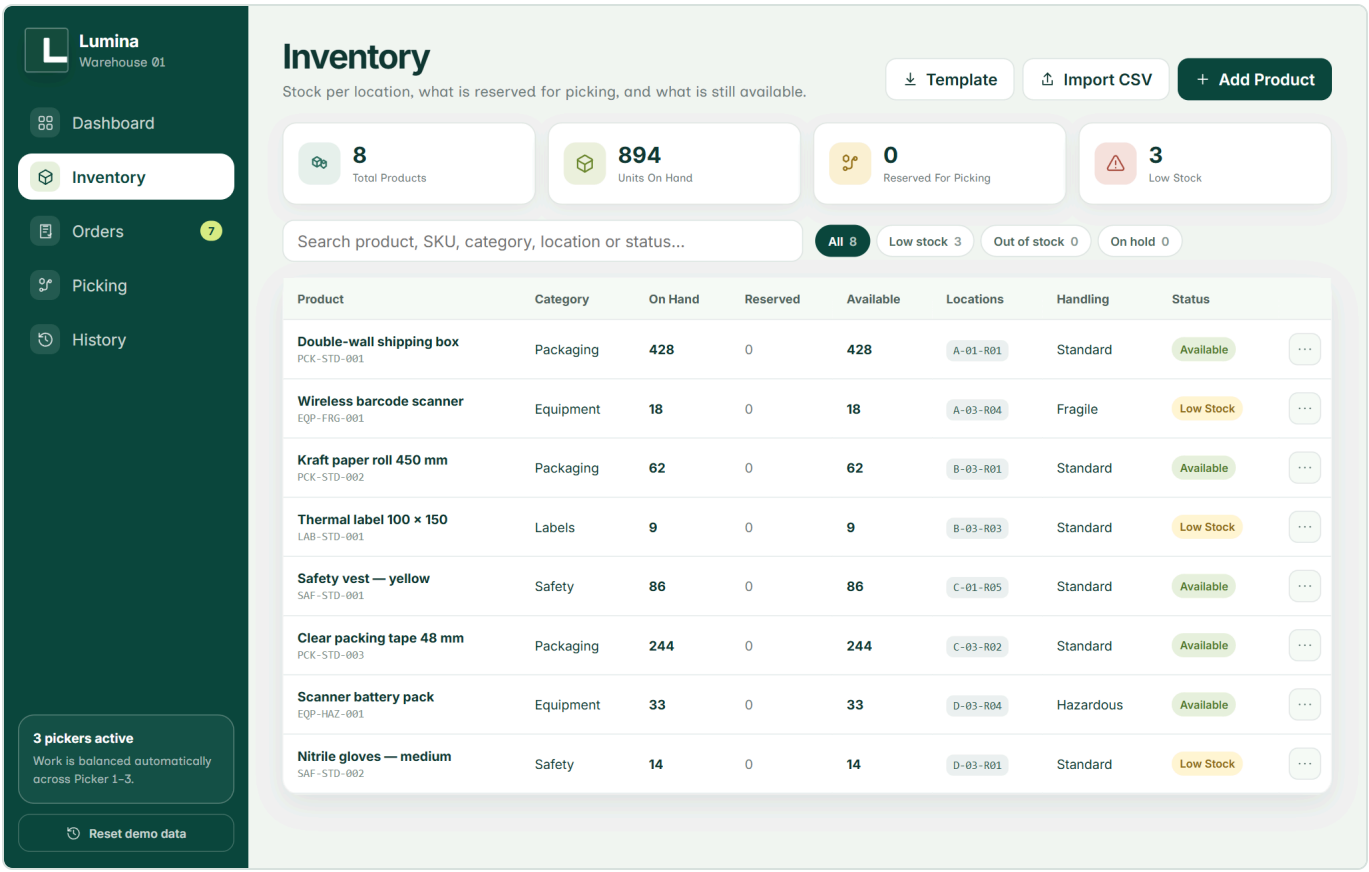


The Dashboard right after starting Lumina with its demo data: 7 orders, 3 low-stock products and 6 notifications.

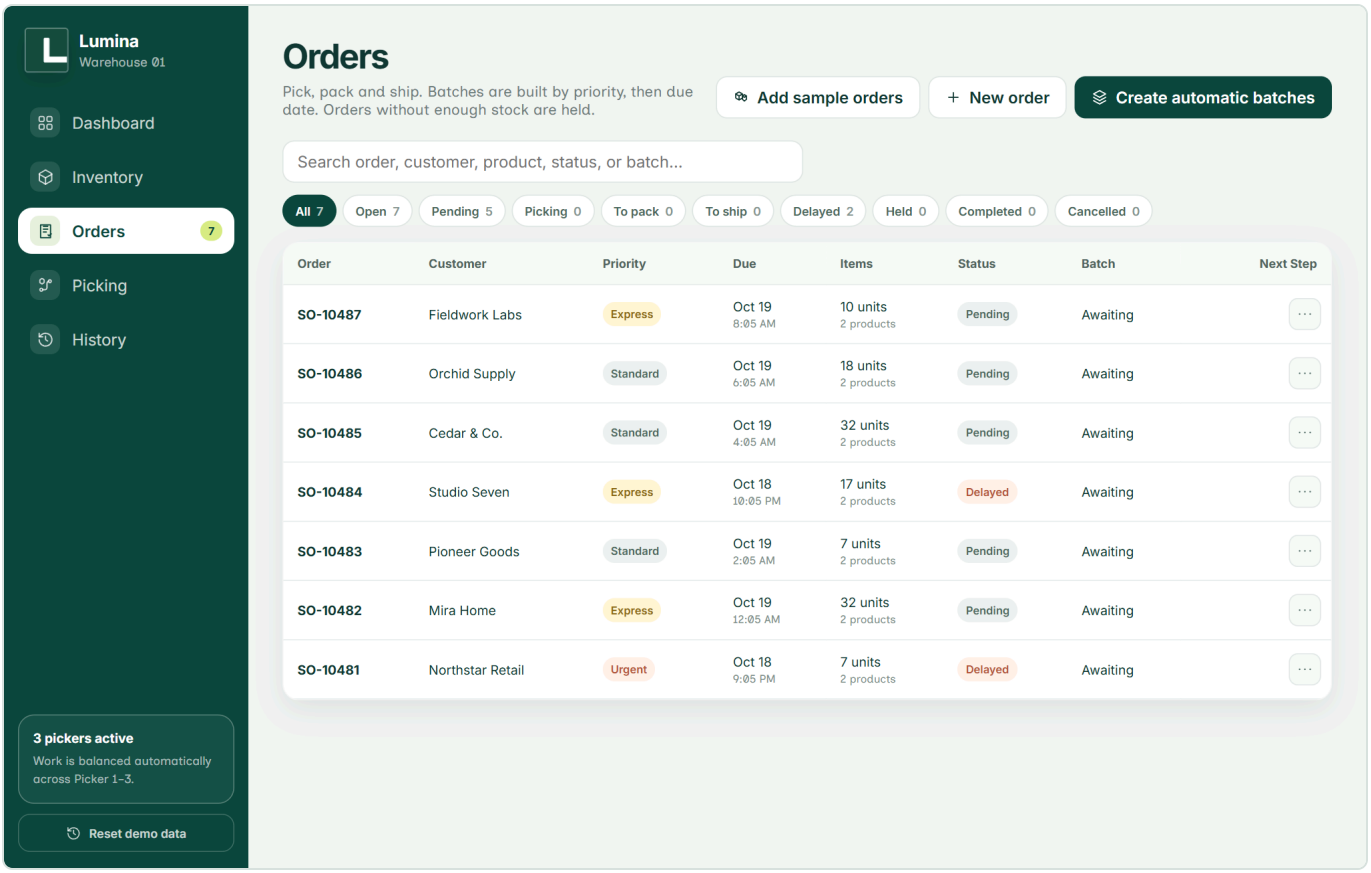
A tour of the five pages

The sidebar has five pages. Each one is drawn by one main function in `renderer.js`.

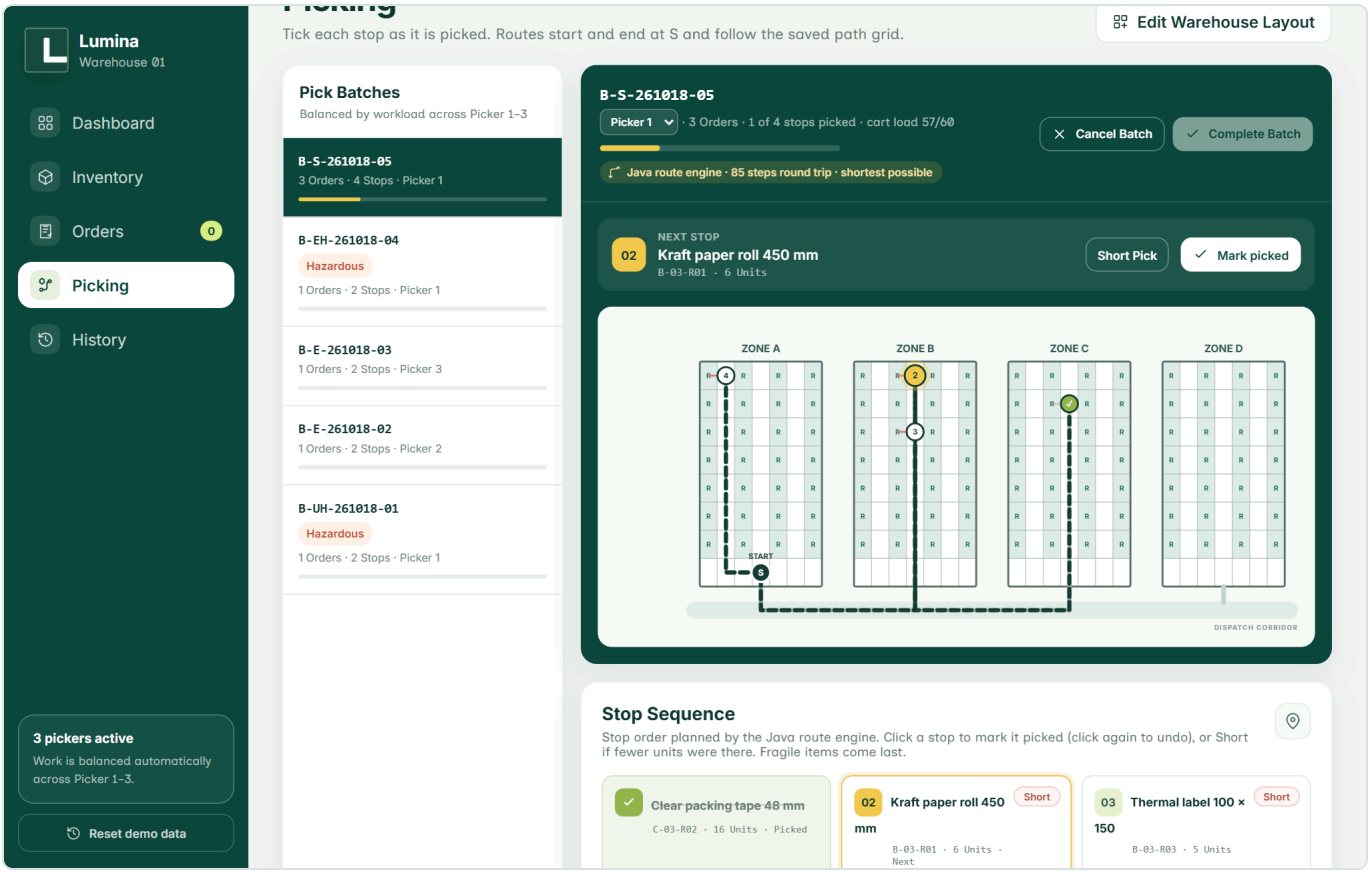
Page	What you do there	Main functions
Dashboard	See the totals (orders, completed, open, low stock), the order chart, the status breakdown and the notifications. Every card and notification is clickable and opens the matching list.	`renderDashboard`, `drawChart`, `buildAlerts`
Inventory	Add and edit products, receive deliveries, move stock between locations, see each product's stock history, download a CSV template and import products from a CSV file.	`renderInventory`, `openProductModal`, `openReceiveModal`, `openLocationsModal`, `importInventoryCsv`
Orders	Create orders (or sample orders), see which are held for lack of stock, create automatic picking batches, then pack and ship picked orders.	`renderOrders`, `openOrderModal`, `createAutomaticBatches`, `packOrder`, `shipOrder`
Picking	Work through a batch stop by stop on the warehouse map, record short picks, complete the batch. Edit the warehouse grid.	`renderPicking`, `toggleStopPicked`, `completeBatch`, `openWarehouseLayoutModal`
History	Read a searchable log of every important change.	`renderHistory`



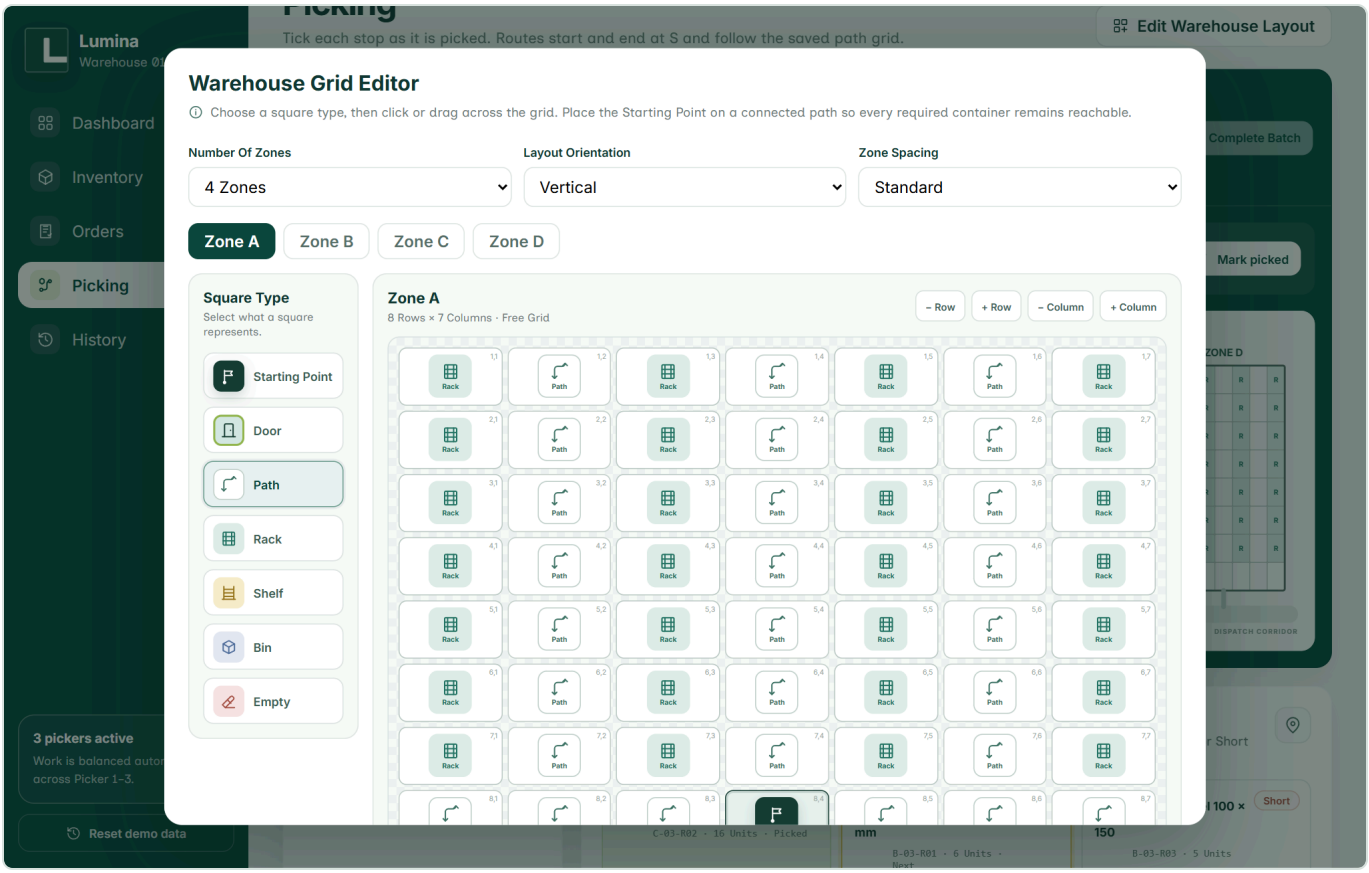
Inventory: stock on hand, how much is reserved for open picking batches, and how much is still available, for each product.



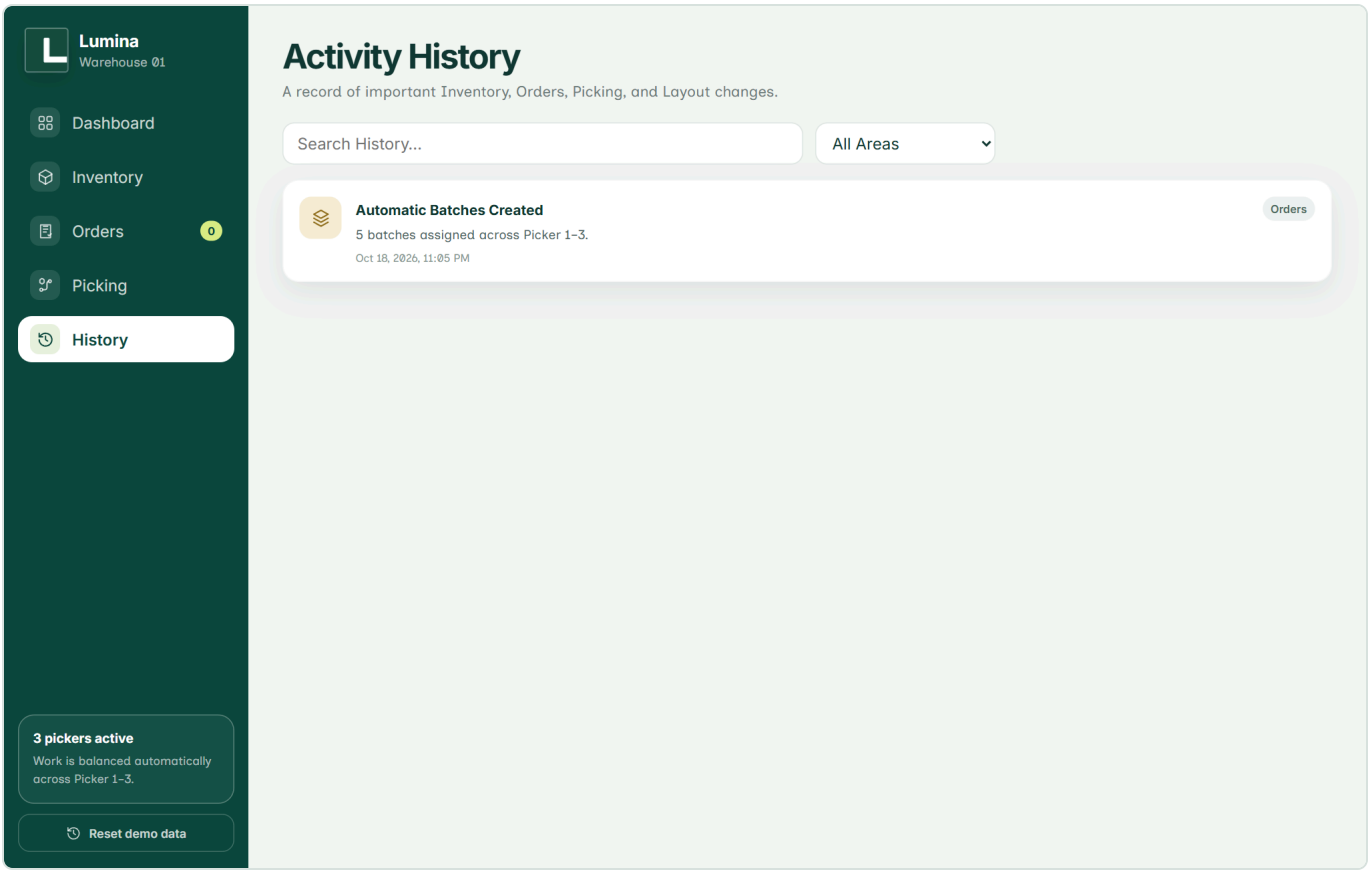
Orders: every order with its priority, due time, status and batch. "Create automatic batches" turns the pending orders into picking batches.



Picking after the first stop was marked picked: the map shows the route from S (start) through the numbered stops and back, the next stop is highlighted, and picked stops get a check mark.



The Warehouse Grid Editor: each zone is a grid of squares. A square can be a Path (walkway), a Rack, a Shelf, a Bin, the Starting Point or a Door to the dispatch corridor.



Activity History: every important action is logged with its area (Inventory, Orders, Picking, Layout).

Technologies used

Technology	Where it is used	What it does in Lumina
Electron	`main.cjs`, `preload.cjs`	Turns the web page into a Windows desktop app. It has a *main process* (Node.js, can start programs) and a *renderer process* (the Chromium window that shows the page).
HTML and CSS	`index.html`, `styles.css`	The page frame (sidebar and an empty `main` area) and all of the styling. Fonts (Inter) are bundled in `fonts/`.
Plain JavaScript	`renderer.js`	Builds every page as HTML text (template strings) and puts it into the page with `innerHTML`, then attaches click handlers. No framework (no React, no Vue).
localStorage	`loadState`, `saveState`	The browser's built-in key-value storage. Lumina saves its whole state as one JSON text under the key `wayline.warehouse.desktop.v1`.
SVG	`drawChart`, `warehouseLayoutSvg`	Vector graphics drawn from code: the order chart and the warehouse map with the route.
Node.js `child_process`	`route-engine.cjs`	Starts the Java engine as a separate program and talks to it through its standard input and output.
Java 21	`lumina-engine.jar`	The route engine. It uses modern Java features: records, switch expressions and pattern matching. No external libraries; it even has its own JSON reader and writer.

Technology	Where it is used	What it does in Lumina
Algorithms	both parts	Breadth-first search (reachability and built-in routing), Dijkstra's shortest paths, Held-Karp dynamic programming (exact route), nearest neighbour plus 2-opt (fast route), Floyd-Warshall (tests), FNV-1a hashing (plan cache key).

The files in the project

File	Size	Role
`Lumina.exe`	97 MB	The built Windows app. Run this to use Lumina.
`Lumina Inventory Import Template.xlsx`	18 KB	A spreadsheet with the columns the CSV import expects.
`Lumina App (source)/package.json`	6 lines	The app's name (`wayline-warehouse-desktop`), version 1.7.0 and its entry file `main.cjs`.
`main.cjs`	43 lines	Electron main process: creates the window, starts the route engine, answers route requests.
`preload.cjs`	6 lines	The safe bridge: gives the page one function, `window.lumina.optimizeRoute`.
`route-engine.cjs`	87 lines	Starts the Java engine and exchanges JSON lines with it.
`renderer.js`	1,376 lines (174 KB)	Everything you see and every business rule: 160 functions and 21 constants or variables.
`index.html`	30 lines	The page frame: sidebar, navigation buttons, `main` area, modal and toast containers.
`styles.css`	485 lines	All styling.
`Lumina Java Engine (source)/src/...`	7 files, 851 lines	The route engine: EngineServer, Json, Warehouse, WarehouseGraph, ShortestPaths, RouteOptimizer, RoutePlan.
`.../test/.../EngineTest.java`	328 lines	Self-checking tests for the engine (not shipped in the app).
`lumina-engine.jar`, `README.txt`		The compiled engine, and how to build and test it.

How to run Lumina

- **The normal way:** double-click `Lumina.exe`. Lumina keeps its data in the folder `%APPDATA%\Wayline Warehouse` (set on line 7 of `main.cjs`, so data from the app's earlier name is kept). The demo data loads the first time; the **Reset demo data** button in the sidebar brings it back at any time.
- **From the source code:** the folder needs Electron (for example `npx electron .` inside `Lumina App (source)`). `package.json` lists no dependencies, so Electron must be installed separately. When the app is not packaged, it looks for the engine in a `java` folder next to `main.cjs` (`main.cjs` line 10); without it, Lumina simply uses its built-in routing.
- **In a web browser:** opening `index.html` directly also works. There is no `window.lumina` in a browser, so routes are planned by the built-in routing only (see the comment on line 1096 of `renderer.js`).
- **The engine's tests:** the commands are in the engine's `README.txt`. For this guide they were run with Java 25: *22,902 checks, 0 failures*; 500 random warehouses matched a brute-force search exactly; a 12-stop exact plan took 4 ms and a 40-stop plan 8 ms.

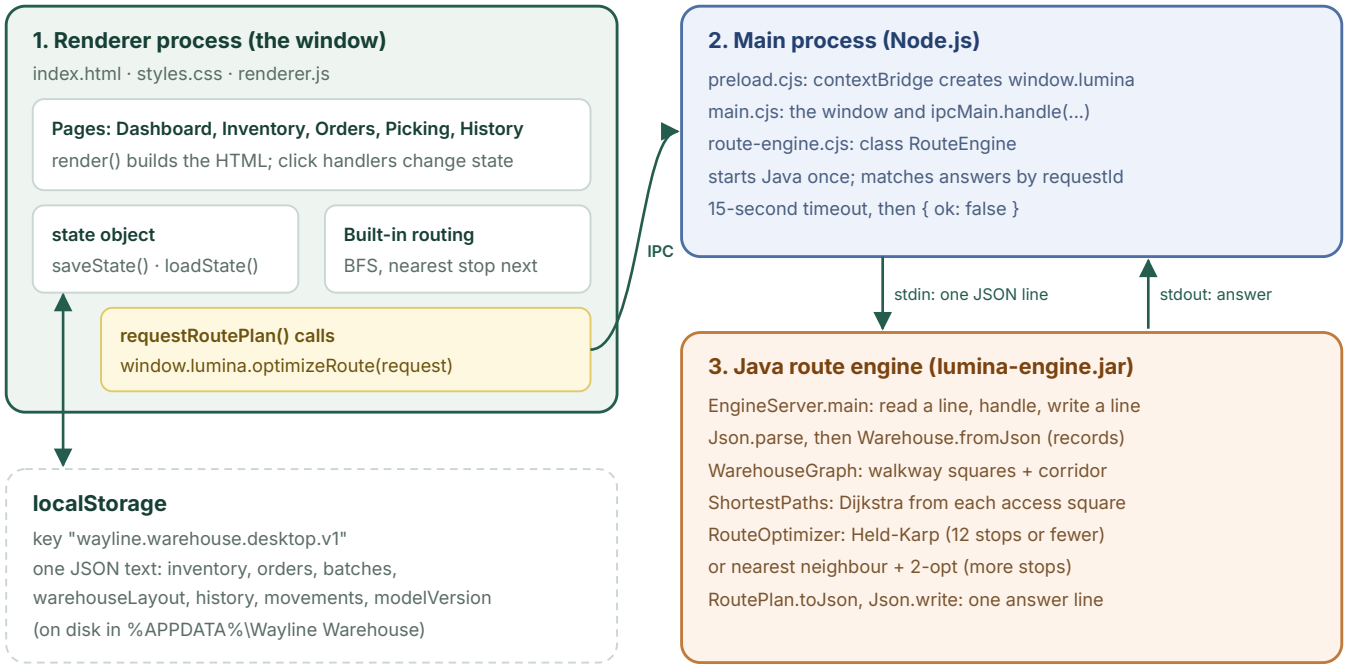
Key numbers and rules at a glance

Rule	Value	Defined in
Units one square can hold	Rack 500, Shelf 250, Bin 100	`CONTAINER_CAPACITY` (renderer.js line 68)
Load one picking cart can carry	60 load units	`CART_CAPACITY` (line 69)
Orders per batch	at most 4	`MAX_ORDERS_PER_BATCH` (line 70)
Pickers	Picker 1, Picker 2, Picker 3	`PICKERS` (line 71)
Oversized items	count 5 load units each	`itemLoad` (line 104)
Hazardous, Refrigerated, Frozen	travel on their own carts	`orderHandlingClass`, `createAutomaticBatches`
Fragile items	always picked last (they ride on top)	`stopIsLast`, `RouteOptimizer.allowed`
Batch order	urgent, then express, then standard; then earliest due	`sortReadyOrders`
Exact route planning	up to 12 stops (Held-Karp)	`RouteOptimizer.EXACT_LIMIT`
Java answer timeout	15 seconds	`RouteEngine.request`
History kept	last 150 entries; stock movements: last 600	`recordHistory`, `recordMovement`

Part 2 · How the pieces fit together

The big picture

Lumina runs as **three programs** that pass messages to each other. The picture below shows them and the files that make up each one.



1. **The renderer process** is the window. It loads `index.html`, which loads `styles.css` and `renderer.js`. Everything the user sees and every business rule is in `renderer.js`. The data lives in one JavaScript object called `state`, saved to localStorage after every change.
2. **The main process** is Node.js. It creates the window (`main.cjs`) and is the only part allowed to start other programs. The page cannot touch Node.js directly; it can only call the one function that `preload.cjs` puts on `window.lumina`.
3. **The Java route engine** is a separate program, started once by `route-engine.cjs` and kept running. It reads one JSON request per line on its standard input and writes one JSON answer per line on its standard output.

Why three programs?

- **Security.** The window is created with `contextIsolation: true`, `nodeIntegration: false` and `sandbox: true` (main.cjs lines 27–32). Web page code therefore cannot read files or start programs. `index.html` also has a Content-Security-Policy that only allows the page's own files. The page gets exactly one extra ability: asking for a route.
- **The right tool for each job.** The page is best at drawing screens. Java is fast at heavy number crunching: the exact route search tries a huge number of combinations in milliseconds.
- **Never stuck.** If Java is missing, crashes or takes longer than 15 seconds, the request simply comes back as `{ ok: false }` and the page keeps its own built-in route. The user sees "Built-in routing · Java engine unavailable".

The data: one `state` object

Everything Lumina knows is in the variable `state` (renderer.js line 151). It is loaded once by `loadState()` and written back by `saveState()` after every change.

Field	Contents	Example
`inventory`	Products. Each has `id`, `sku` (product code), `name`, `category`, `quantity` (total), `reorderPoint` (low-stock threshold), `handlingType`, `status`, and `locations`: a list of `{ location, quantity }`. The first location is the *primary* one.	Double-wall shipping box, SKU `PCK-STD-001`, 428 units at `A-01-R01`
`orders`	Customer orders: `orderNumber`, `customer`, `priority` (urgent / express / standard), `dueAt`, `status`, `batchCode`, `lines` (`{ sku, quantity }`), `events` (time of each step), `shortPicked`.	`SO-10481`, Northstar Retail, urgent
`batches`	Picking batches: `code`, `status` (planned / active / complete), `orderIds`, `route` (the stops), `assignedPicker`, `progress`, `handlingClass`, `load`, and the Java `plan` when there is one.	`B-S-261018-04`, 3 orders, 4 stops, Picker 1
`warehouseLayout`	The warehouse grid: `zoneCount`, `orientation`, `aisleWidth`, `startingPoint`, `doors`, and `zoneLayouts` (one grid per zone A–D).	4 zones of 8 rows × 7 columns, start at zone A square 8-4
`history`	Activity log entries: `action`, `detail`, `area`, `icon`, `at`.	"Batch Completed"
`movements`	Stock movements: `sku`, `location`, `change`, `balance`, `type` (Received, Picked, Transfer, Moved, Adjustment, Count correction…), `reference`.	+20 Received at A-01-R01, PO-2041
`modelVersion`	The data format version (17), used to upgrade older saved data.	17

Location codes and grid squares

Every storage square has a **location code** made by `composeLocation` and read by `parseLocation`:

renderer.js · lines 632–632

```
632  function composeLocation(data) { const pad = (value) => String(value).padStart(2,"0"); const code = { Rack:"R", Shelf:"S", Bin:"B" }[data.containerType] || "R"; return `${String(data.zone).toUpperCase()}-${pad(data.aisle)}-${code}${pad(data.containerNumber)}`; }
```

For example `A-03-R04` means *zone A, grid column 3, a Rack, grid row 4*. (In the code the column is called `aisle` and the row `containerNumber`, names left over from older versions.) Inside a zone, squares are also identified by a **grid key** `"row-column"`, for example `"8-4"`, made by `gridKey(row, column)`.

The default zone (made by `createZoneLayout`) has 8 rows and 7 columns. The bottom row and every even column are **Path** squares (walkways); the odd columns of rows 1–7 are **Racks**:

renderer.js · lines 51–51

```
51  function createZoneLayout(rows = 8, columns = 7, type = "Rack") { const cells = {}; for (let row = 1; row <= rows; row += 1) for (let column = 1; column <= columns; column += 1) cells[`${row}-${column}`] = row === rows || column % 2 === 0 ? "Path" : type; return { rows, columns, cells }; }
```

A picker can only walk on Path squares, never through a Rack, Shelf or Bin. To pick from a rack the picker stands on a Path square directly above, below, left or right of it. The zones are joined by a **dispatch corridor** along their bottom edge (or left edge when the layout is horizontal). Each zone has one **door** onto the corridor. The **starting point S** is where every route starts and ends (the cart drop-off). A location is **reachable** when a walk exists from S, through the doors and corridor if needed, to a Path square next to it.

Business rules in the code

- **Capacity.** A Rack holds 500 units, a Shelf 250 and a Bin 100. Each square holds one product only (`locationProblem`).
- **Reserved stock.** Units in open picking batches are *reserved*: still on the shelf, but promised. Inventory shows On hand, Reserved and Available = On hand − Reserved (`reservedBySku`).
- **Holding orders.** An order is *held* (not batched) when one of its products is missing, damaged, quarantined, out of stock, unreachable, or short. Orders are checked in priority order, so urgent orders get the stock first (`heldOrders`, `orderBlockers`).
- **Carts.** One batch is one cart: at most 4 orders and 60 load units. Oversized items count 5 load units each. Hazardous goods and cold-chain goods (Refrigerated, Frozen) get their own carts. Orders on one cart also share priority and due day (`createAutomaticBatches`).
- **Fair work.** Each new batch goes to the picker with the least unfinished work, measured in load units, not number of batches.
- **Fragile last.** Fragile items are picked after everything else so they ride on top of the cart (`stopIsLast`, and the Java `allowed` rule).
- **Short picks.** If a picker finds fewer units than expected, the shelf count is corrected when the batch is completed, and orders that can't be filled go back to Pending (`openShortPickModal`, `completeBatch`).

The life cycle of an order

Stored `status`	Shown as	How it gets there	Function
`ready`	Pending (or Delayed, Held)	Order created	`addOrder`, `generateSampleOrders`
`batched`	Picking	Put on a cart	`createAutomaticBatches`
`picked`	Picked (To pack)	Its batch was completed with enough stock	`completeBatch`
`packed`	Packed (To ship)	Pack button	`packOrder`
`shipped`	Completed	Ship button	`shipOrder`
`cancelled`	Cancelled	Cancel button (only while ready or batched)	`cancelOrder`

An open order whose due time has passed is shown as **Delayed** (`orderDisplayStatus`). A batched order can also go back to `ready`: when it is removed from its batch, when the batch is cancelled, or when a short pick leaves too little stock for it.

How a picking route is planned

1. `createAutomaticBatches` turns orders into stops with `allocateStops` and orders them right away with the **built-in routing** (`optimizeRouteStops`, breadth-first search, nearest stop next).
2. When the Picking page shows a batch, `renderPicking` checks `hasValidPlan`. A saved Java plan is valid only while the layout and the stops are unchanged (`routePlanKey`).
3. If there is no valid plan, `requestRoutePlan` sends `buildRouteRequest(batch)` through `window.lumina.optimizeRoute`, to `ipcMain` in `main.cjs`, then `RouteEngine.request`, to the Java process.

4. Java answers with the best visiting order, the walking path of every leg, the distance and the built-in route's distance for comparison. The page reorders the stops, saves the plan and redraws the map, with the chip "Java route engine · N steps round trip · shortest possible".

Saving and upgrading data

`saveState()` writes the whole state as JSON after every change. When Lumina starts, `loadState()` reads it back and runs the **migrations**: older versions stored warehouses differently (two-sided aisles, older location codes, older status names), so functions such as `migrateLocationToFreeGrid` and `upgradeState` convert old data to today's format. This is why a warehouse saved by an old version still opens correctly.

Part 3 · The desktop shell: Electron files

main.cjs: the main process

`main.cjs` is the first file Electron runs (`package.json` names it as `"main"`). It is only 43 lines long. It names the app, decides where data and the Java engine live, creates the window, and answers route requests from the page.

main.cjs · lines 1–16

```
1  const { app, BrowserWindow, ipcMain } = require("electron");
2  const path = require("path");
3  const { RouteEngine } = require("./route-engine.cjs");
4
5  app.setName("Lumina");
6  // Keep using the data folder from earlier versions (%APPDATA%\Wayline Warehouse) so nothing is lost after the
   rename.
7  app.setPath("userData", path.join(app.getPath("appData"), "Wayline Warehouse"));
8
9  // The Java route engine and its runtime live outside the app archive, in resources/java.
10 const engineDir = app.isPackaged ? path.join(process.resourcesPath, "java") : path.join(__dirname, "java");
11 const routeEngine = new RouteEngine(engineDir);
12
13 ipcMain.handle("route:optimize", async (_event, request) => {
14   if (!request || typeof request !== "object") return { ok: false, error: "Invalid route request" };
15   return routeEngine.request({ ...request, type: "optimize" });
16 });
```

- **Lines 1–3** load Electron's `app` (the application), `BrowserWindow` (a window) and `ipcMain` (messages from the page), Node's `path` helper, and the `RouteEngine` class from `route-engine.cjs`.
- **Line 5** sets the app name to Lumina.
- **Line 7** keeps the data folder from the app's earlier name, *Wayline Warehouse* (`%APPDATA%\Wayline Warehouse`), so updating or renaming the app never loses the user's data. localStorage is stored inside this folder.
- **Line 10** works out where the Java engine is. In the installed app (`app.isPackaged`) it sits in the `resources/java` folder next to the executable, outside the app archive, because a program cannot be started from inside an archive. While developing, it is a `java` folder next to `main.cjs`.
- **Line 11** creates one `RouteEngine` for the whole app. Java is not started yet; it starts on the first request.
- **Lines 13–16** register the handler for the message `route:optimize`. When the page asks for a route, this checks that the request is an object, adds `type: "optimize"`, and passes it to `routeEngine.request(...)`. The promise it returns is sent back to the page automatically.

`createWindow()`

main.cjs · line 18

Creates the application window: 1360 × 860 pixels (at least 1040 × 700), a light green-grey background while loading, no menu bar, and the Lumina icon. The `webPreferences` block is the security setting: `contextIsolation` and `sandbox` keep the page separate from Node.js, `nodeIntegration: false` means page scripts cannot use `require`, and `preload` names the one script allowed to build a bridge (`preload.cjs`). Finally `win.loadFile("index.html")` shows the page.

main.cjs · lines 37–43

```
37 app.whenReady().then(() => {
38   createWindow();
39   app.on("activate", () => { if (BrowserWindow.getAllWindows().length === 0) createWindow(); });
40 });
41
42 app.on("will-quit", () => routeEngine.stop());
43 app.on("window-all-closed", () => { if (process.platform !== "darwin") app.quit(); });
```

- **Lines 37–40** create the window once Electron is ready. On macOS (`activate`) a new window is created when the dock icon is clicked and no window is open.
- **Line 42** stops the Java process when the app quits (`routeEngine.stop()`), so no Java program is left running in the background.
- **Line 43** quits when all windows are closed, except on macOS where apps usually stay open.

Called from: `app.whenReady()`

preload.cjs: the safe bridge

preload.cjs · lines 1–6

```
1  // Exposes a single, narrow function to the page: ask the Java engine to plan a route.
2  const { contextBridge, ipcRenderer } = require("electron");
3
4  contextBridge.exposeInMainWorld("lumina", {
5    optimizeRoute: (request) => ipcRenderer.invoke("route:optimize", request),
6  });
```

The preload script runs before the page, with access to a small part of Electron. `contextBridge.exposeInMainWorld("lumina", …)` creates the object `window.lumina` in the page with a single function, `optimizeRoute(request)`. It forwards the request to the main process with `ipcRenderer.invoke("route:optimize", request)` and returns a promise for the answer. The page never gets `ipcRenderer` itself, only this one function. This is the "principle of least privilege": give each part only what it needs.

route-engine.cjs: talking to Java

This file defines the class `RouteEngine`, which owns the Java process. Requests and answers are matched by a number, the `requestId`, because several requests could be waiting at the same time.

`class RouteEngine` route-engine.cjs · line 8

A class keeps the Java process and its bookkeeping together. One instance is created in `main.cjs` and lives as long as the app.

Called from: `routeEngine (main.cjs)`

`constructor(engineDir)` route-engine.cjs · line 9

Saves the folder that holds the engine and sets the starting values: `child` (the Java process, `null` until started), `buffer` (text received but not yet a full line), `pending` (a `Map` from requestId to the waiting promise), `nextRequestId` (starts at 1) and `lastError`.

`javaCommand()` route-engine.cjs · line 18

Chooses which Java to run. The installed app bundles its own Java runtime in `java/runtime/bin/java.exe` (`java` on Mac and Linux). If that file exists it is used, so the user does not need Java installed. Otherwise it falls back to plain `java`, a Java found on the system PATH.

Called from: `start`

`start()`

route-engine.cjs · line 24

Starts the Java engine once. If a process is already running it does nothing. It checks that `lumina-engine.jar` exists (otherwise it throws "Route engine file is missing"), then runs `java -Xmx256m -XX:TieredStopAtLevel=1 -Xshare:auto -jar lumina-engine.jar`. These options limit memory to 256 MB and make Java start faster (quick compilation and shared class data), which suits a small helper program. `windowsHide: true` stops a console window from appearing.

The `stdout` handler collects text in `buffer`. Data can arrive in pieces, so it cuts the buffer at each newline and passes every complete line to `receive`. Errors written by Java (`stderr`) and broken-pipe errors are ignored. When the process fails to start or exits, `fail` is called with a readable reason.

Called from: `request`

`receive(line)`

route-engine.cjs · line 49

Handles one answer line. It parses the JSON (a broken line is ignored), finds the waiting request with the same `requestId`, removes it from `pending`, cancels its timeout and resolves its promise with the answer.

Called from: `start`

`fail(child, reason)`

route-engine.cjs · line 59

Called when the Java process cannot start or stops. The first check, `this.child !== child`, ignores events from an old process that was already replaced. It forgets the process, clears the buffer, remembers the reason, and resolves **every** waiting request with `{ ok: false, error: reason }`, so the page is never left waiting. The next request will start a new Java process.

Called from: `start`

`request(payload, timeoutMs = 15000)`

route-engine.cjs · line 68

Sends one request and returns a promise for the answer.

1. Calls `start()`. If that throws (for example the jar is missing), the promise resolves at once with `{ ok: false, error }`.
2. Takes the next `requestId` and starts a 15-second timer. If Java has not answered by then, the request is removed and resolved with "Java engine did not answer in time".
3. Stores `{ resolve, timer }` in `pending` and writes the request plus its `requestId` as one JSON line to Java's standard input.

The promise always *resolves* (never rejects), with either Java's answer or an `ok: false` object. That keeps the page's code simple.

Called from: `ipcMain.handle(route:optimize) (main.cjs)`

`stop()`

route-engine.cjs · line 78

Closes Java's input and kills the process. Each step is wrapped in `try` because the process may already have ended. Called from `main.cjs` when the app quits.

> **Note.** `module.exports = { RouteEngine }` on the last line makes the class available to `main.cjs` through `require("./route-engine.cjs")`. The `.cjs` extension tells Node.js these are CommonJS modules (the `require` style).

Called from: `app.on(will-quit) (main.cjs)`

index.html: the page frame

index.html · lines 10–29

```
10  <body>
11    <div class="shell">
12      <aside class="sidebar">
13        <div class="brand"><img class="brand-logo" src="build/logo-mark.png" alt="Lumina logo" /><div>
    <strong>Lumina</strong><span>Warehouse 01</span></div></div>
14        <nav aria-label="Main navigation">
15          <button class="nav-item active" data-view="dashboard"><span class="nav-icon" data-icon="dashboard">
    </span>Dashboard</button>
16          <button class="nav-item" data-view="inventory"><span class="nav-icon" data-icon="inventory">
    </span>Inventory</button>
17          <button class="nav-item" data-view="orders"><span class="nav-icon" data-icon="orders"></span>Orders <b
    id="ready-badge">0</b></button>
18          <button class="nav-item" data-view="picking"><span class="nav-icon" data-icon="picking">
    </span>Picking</button>
19          <button class="nav-item" data-view="history"><span class="nav-icon" data-icon="history">
    </span>History</button>
20        </nav>
21        <div class="picker-card"><strong>3 pickers active</strong><span>Work is balanced automatically across
    Picker 1–3.</span></div>
22        <button class="reset-demo" id="reset-demo" type="button"><span data-icon="history"></span>Reset demo
    data</button>
23      </aside>
24      <main id="app" tabindex="-1"></main>
25    </div>
26    <div id="modal-root"></div>
27    <div id="toast" role="status" aria-live="polite"></div>
28    <script src="renderer.js"></script>
29  </body>
```

`index.html` only provides the frame. The **sidebar** has the brand, five navigation buttons (each with `data-view`, for example `data-view="orders"`), the picker card and the **Reset demo data** button. The `<b id="ready-badge">` inside the Orders button shows how many orders are pending. The ** `main id="app"` ** element is empty: `renderer.js` fills it with the current page. `#modal-root` holds pop-up windows (modals), and `#toast` shows short messages at the bottom right (`aria-live="polite"` makes screen readers announce them). Elements with `data-icon` get their SVG icons from `hydrateStaticIcons()`.

The Content-Security-Policy on line 6 allows scripts and styles only from the app's own files, plus inline styles and `data:` images. This blocks code from other websites.

styles.css: the look

`styles.css` (485 lines) holds all of the styling. A few parts are worth knowing:

- **Lines 1–4** load the bundled fonts (Inter and Inclusive Sans) from the `fonts` folder, so the app looks the same on every computer without internet.
- **Lines 6–18** define colour variables in `:root` (`--green: #0b493d`, `--page`, `--line`, and soft card colours `--cyan`, `--lime`, `--yellow`, `--rose`). Every rule uses these variables, so the palette can be changed in one place.
- The **layout** is a CSS grid: a fixed sidebar and the main area. Tables, cards, pills (status badges), modals and the toast each have their own rules.
- The **zone grid editor** uses `grid-template-columns: repeat(var(--board-columns), …)` (line 276). `renderZoneEditor` sets `--board-columns` to the zone's number of columns, so the CSS grid always matches the warehouse grid.
- ** `@media` ** rules (lines 319 and 328) rearrange the dashboard and tables on narrower windows.

- Two **animations**: `row-flash` (line 364) briefly highlights the row you jumped to from a notification (see `applyHighlight`), and `engine-pulse` (line 418) makes the "Java engine is optimizing" chip pulse while waiting.

Part 4A · renderer.js: setup, data and rules

`renderer.js` is one long script loaded at the end of `index.html`. It has no imports: every function is declared at the top level, so any function can call any other. The file is organized in this order: setup and icons, the warehouse model and data upgrades, stock rules, the saved state, shared helpers, navigation, then one block per page (Dashboard, Inventory, Orders, Picking, History), routing, the map, and the layout editor. The last six lines start the app.

Each entry below shows the function's signature, where it is (line number in `renderer.js`), what it does, how it works, and which functions call it. "Called from" was worked out from the code itself.

Setup and icons

`STORAGE_KEY` renderer.js · line 1

The name under which the whole state is saved in localStorage: `"wayline.warehouse.desktop.v1"`. The name comes from the app's earlier name (Wayline Warehouse) and is kept so existing data still loads.

`app` renderer.js · line 2

The `<main id="app">` element. Every page is drawn by setting `app.innerHTML`.

`modalRoot` renderer.js · line 3

The `#modal-root` element. Pop-up windows (modals) are drawn into it; emptying it closes the modal.

`toastElement` renderer.js · line 4

The `#toast` element used by `toast()` for short messages.

`ICON_PATHS` renderer.js · line 6

An object that maps an icon name (`dashboard`, `inventory`, `orders`, `picking`, `rack`, `door`, `path`, …) to the inside of an SVG drawing (paths, circles, rectangles). Keeping icons as text in the code means no image files are needed and icons take the text colour.

`icon(name, className = "ui-icon")` renderer.js · line 42

Returns the HTML for one icon: an `<svg>` element with a 24 × 24 viewBox, drawn with lines (`stroke="currentColor"`, so it uses the surrounding text colour) and marked `aria-hidden` because icons are decoration. If the name is unknown it falls back to the `package` icon. It is used all over the page builders, for example `icon("check")` inside buttons.

Called from: `hydrateStaticIcons`, `renderDashboard`, `kpi`, `alertMarkup`, `renderInventory`, `productRow`, `openProductModal`, `openReceiveModal`, `openLocationsModal`, `openStockHistoryModal`, `renderOrders`, `orderActionsMarkup`, `openOrderModal`, `openOrderDetails` and 6 more

`hydrateStaticIcons()` renderer.js · line 46

Fills every element in `index.html` that has a `data-icon` attribute (the sidebar navigation and the reset button) with its SVG icon. Called once at startup.

Called from: `startup code`

`due(hours)`

renderer.js · line 50

A one-line arrow function: the date and time `hours` from now, as ISO text. A negative number gives a time in the past. The demo orders use it, so their due times are always relative to when the data was created: `due(-2)` makes order SO-10481 already two hours late, so it shows as Delayed.

Called from: `defaultState`

The warehouse model and data upgrades

These functions build, check and upgrade the warehouse grid. Version 3 of the layout (the current one, called the *free grid*) lets every square be any type. Versions 1 and 2 used racks along aisles, so there are converters for data saved by those versions.

`createZoneLayout(rows = 8, columns = 7, type = "Rack")`

renderer.js · line 51

Builds the default grid for one zone: an object `{ rows, columns, cells }` where `cells` maps each grid key `"row-column"` to a square type. A square is `"Path"` if it is in the last row or an even column; otherwise it is `type` (Rack by default). The result is aisles of racks with walkways between them and a walkway along the bottom, next to the dispatch corridor.

Called from: `defaultWarehouseLayout`, `normalizeWarehouseLayout`, `openProductModal`

`normalizeZoneToTwoSides(source, defaultType)`

renderer.js · line 52

Upgrades a zone saved in the oldest (version 1) format to the version 2 format, where every row has a container on the left side and the right side of a single aisle. Old zones with more than two columns are "unfolded": each old square gets a running index `(row − 1) × columns + column − 1`, and the index is turned into a new row (`index / 2`) and side (`index % 2`). Only Rack, Shelf and Bin squares are kept, and the number of rows is limited to 2–48.

Called from: `convertTwoSideZoneToGrid`

`convertTwoSideZoneToGrid(source, defaultType)`

renderer.js · line 53

Turns a version 2 (two-sided) zone into today's free grid with three columns: the left container in column 1, a Path in column 2 (the aisle), and the right container in column 3. It first calls `normalizeZoneToTwoSides`, so it also handles version 1 data.

Called from: `normalizeWarehouseLayout`

`normalizeGridZone(source, defaultType)`

renderer.js · line 54

Cleans a free-grid zone: rows and columns are limited to 3–30, and only squares inside the grid with a valid type (Path, Rack, Shelf, Bin) are kept. Anything else in the saved data is dropped, so a damaged save cannot break the map.

Called from: `normalizeWarehouseLayout`

`defaultWarehouseLayout()`

renderer.js · line 55

The demo warehouse: 4 zones (A–D), vertical orientation, standard spacing, Racks as the default container, format versions 3, starting point in zone A at square `8-4` (bottom row, middle), and four zones made by `createZoneLayout()`.

Called from: `normalizeWarehouseLayout`, `defaultState`

`normalizeWarehouseLayout(saved = {})` renderer.js · line 56

Makes any saved layout safe to use. It is called before almost every use of the layout, so the rest of the code can trust it.

1. Starts from the default layout and keeps the number of zones between 2 and 4 and the container type among Rack, Shelf and Bin.
2. For each zone A–D: no saved zone gives a default zone; free-grid data (version 3) is cleaned with `normalizeGridZone`; older data is converted with `convertTwoSideZoneToGrid`.
3. Checks the starting point. It must be in an active zone and on a Path square. If not, it moves to zone A at the zone's entry square (`zoneEntryPath`), or the first Path square, and makes sure that square is a Path.
4. Returns the saved settings merged over the defaults, with the version numbers set to 3.

Called from: `loadState`, `renderPicking`, `isContainerReachable`, `buildEfficientRoutePlan`, `buildRouteRequest`, `warehouseLayoutSvg`, `openWarehouseLayoutModal`

`migrateLocationToTwoSides(value, savedLayout = {})` renderer.js · line 62

Converts one old location code to the version 2 layout. Very old codes like `A-01-R01-S01-B01` (zone, aisle, rack, shelf, bin) are recognized by a regular expression. If the saved layout is already version 2 or later, or its zones had at most two columns, only legacy codes are rewritten to the current format. Otherwise the same unfolding as `normalizeZoneToTwoSides` moves the location to its new row and side.

Called from: `migrateLocationToFreeGrid`

`migrateLocationToFreeGrid(value, savedLayout = {})` renderer.js · line 63

Converts a location code to the free grid (version 3). If the layout is already version 3 the code is unchanged. Otherwise it first applies `migrateLocationToTwoSides`, then maps side 2 to column 3 and side 1 to column 1, matching `convertTwoSideZoneToGrid`.

Called from: `migrateInventoryToFreeGrid`, `migrateBatchesToFreeGrid`

`migrateInventoryToFreeGrid(inventory, savedLayout = {})` renderer.js · line 64

Applies `migrateLocationToFreeGrid` to every product's location.

Called from: `loadState`

`migrateBatchesToFreeGrid(batches, savedLayout = {})` renderer.js · line 65

Applies `migrateLocationToFreeGrid` to every stop of every picking batch, so old batches still point to the right squares.

Called from: `loadState`

`migrateInventoryStatuses(inventory)` renderer.js · line 66

Renames old product statuses: `Active` becomes `Available`, `Hold` becomes `Quarantined` and `Out of stock` becomes `Out Of Stock`. Missing statuses become `Available`.

Called from: `loadState`

Stock, cart and batching rules

`CONTAINER_CAPACITY` renderer.js · line 68

`{ Rack: 500, Shelf: 250, Bin: 100 }`: how many units one square of each type can hold.

CART_CAPACITY renderer.js · line 69

60: the load units one picking cart can carry.

MAX_ORDERS_PER_BATCH renderer.js · line 70

4: the most orders that share one cart.

PICKERS renderer.js · line 71

`["Picker 1", "Picker 2", "Picker 3"]`: the team. Batches are shared among them.

MODEL_VERSION renderer.js · line 72

17: the version of the saved data format. `upgradeState` uses it to know when old data needs converting.

productBySku(sku) renderer.js · line 73

Finds the product with this product code (SKU) in `state.inventory`, or `undefined`.

Called from: `orderHandlingClass`, `itemLoad`, `locationProblem`, `orderBlockers`, `orderRow`, `openOrderDetails`, `allocateStops`, `completeBatch`

containerTypeOf(location) renderer.js · line 74

The container type (Rack, Shelf or Bin) written in a location code, using `parseLocation`.

Called from: `capacityOf`, `locationProblem`, `openLocationsModal`

capacityOf(location) renderer.js · line 75

How many units the square of a location can hold, from `CONTAINER_CAPACITY` (0 for an unknown type).

Called from: `buildAlerts`, `locationProblem`, `locationOptionLabel`, `openReceiveModal`, `openLocationsModal`, `importInventoryCsv`

syncProduct(item) renderer.js · line 77

Keeps a product's numbers consistent. A product stores its stock per location in `item.locations`; the rest of the app also reads `item.quantity` (the total) and `item.location` (the primary location). This function removes entries without a location, rounds every quantity to a whole number of at least 0, builds `locations` from `location`/`quantity` for products saved before multi-location stock existed, then sets `quantity` to the sum and `location` to the first entry. It updates the entries in place, so code that already holds a reference to an entry sees the new numbers. It returns the item, so it can be used inside expressions.

Called from: `normalizeProduct`, `openProductModal`, `openReceiveModal`, `openLocationsModal`, `importInventoryCsv`, `completeBatch`, `openWarehouseLayoutModal`

normalizeProduct(item) renderer.js · line 86

Makes sure an older product has a `locations` list (from its single location and quantity), then calls `syncProduct`.

Called from: `upgradeState`

`locationOwners(exceptSku = null)`

renderer.js · line 88

Builds a `Map` from every used location code to the name of the product stored there (one product per square), optionally ignoring one product. It is used to grey out occupied squares on the map and to refuse a square that already holds something else.

Called from: `locationProblem`, `openLocationsModal`, `importInventoryCsv`, `updateLocationPreview`

`recordMovement(item, location, change, type, reference = "")`

renderer.js · line 89

Writes one line into the stock history (`state.movements`): when, which product, which location, the change (+ or −), the new total (`balance`), the type of movement and an optional reference such as a purchase order number. A zero change is skipped, except for a count correction. New lines go at the front and only the newest 600 are kept.

Called from: `openProductModal`, `openReceiveModal`, `openLocationsModal`, `importInventoryCsv`, `completeBatch`

`reservedBySku()`

renderer.js · line 95

How many units of each product are promised to open picking batches: it adds up the stop quantities of every batch that is not complete. Inventory shows these as "Reserved".

Called from: `renderInventory`

`reservedByLocation(exceptBatchId = null)`

renderer.js · line 96

The same reservation, but per product *and* location, with keys like `"PCK-STD-001@A-01-R01"`. It can ignore one batch, which is needed when that batch's own stops are being recalculated.

Called from: `openLocationsModal`, `pickableStockBySku`, `allocateStops`

`orderHandlingClass(order)`

renderer.js · line 98

Classifies an order for cart sharing: `"Hazardous"` if any product is hazardous, otherwise `"Cold chain"` if any product is refrigerated or frozen, otherwise `"Standard"`.

Called from: `openOrderDetails`, `batchHandlingClass`, `createAutomaticBatches`

`itemLoad(sku, quantity)`

renderer.js · line 104

The cart space taken by a quantity of a product: the quantity itself, or five times the quantity for oversized products.

Called from: `orderLoad`, `batchLoad`

`orderLoad(order)`

renderer.js · line 105

The total cart load of an order: the sum of `itemLoad` over its lines.

Called from: `openOrderDetails`, `createAutomaticBatches`, `detachOrderFromBatch`

`batchLoad(batch)`

renderer.js · line 106

The total cart load of a batch: the sum of `itemLoad` over its stops.

Called from: `createAutomaticBatches`, `renderPicking`

`ORDER_STAGE_LABEL` renderer.js · line 108

Maps each stored order status to the word shown on screen: ready → Pending, batched → Picking, picked → Picked, packed → Packed, shipped → Completed, cancelled → Cancelled.

`orderStage(order)` renderer.js · line 109

The on-screen stage of an order, using `ORDER_STAGE_LABEL` (Pending if the status is unknown).

Called from: `orderDisplayStatus`, `buildAlerts`, `orderRow`, `renderPicking`

`isOrderOpen(order)` renderer.js · line 110

True while an order still needs work, meaning it is not shipped and not cancelled.

Called from: `isOrderDelayed`, `orderDisplayStatus`, `renderDashboard`, `renderOrders`, `orderRow`

`upgradeState(saved)` renderer.js · line 111

Brings saved data up to the current format (version 17):

- normalizes every product (`normalizeProduct`) and makes sure there is a `movements` list;
- for data saved before the version number existed (before Lumina 1.7), the status `picked` meant "finished", so such orders become `shipped`;
- gives every order an `events` object (with at least `createdAt`), every batch stop an `id` (`sku@location`), and every batch a valid picker (Picker 1 if the saved one is unknown).

Called from: `loadState`, `resetDemoData`

The saved state

`defaultState()` renderer.js · line 126

Returns the demo data used on first start and after **Reset demo data**: 8 products (packaging, equipment, labels and safety items, including a Fragile barcode scanner and a Hazardous battery pack), 7 orders (SO-10481 to SO-10487) with different priorities and due times, no batches, the default warehouse layout and an empty history. Several products are below their reorder point on purpose, so the dashboard has something to show.

Called from: `loadState`, `resetDemoData`

`state` renderer.js · line 151

The single object holding all of Lumina's data (see Part 2). It is set at startup by `loadState()` and replaced only by `resetDemoData()`.

`currentView` renderer.js · line 152

Which page is shown: `"dashboard"`, `"inventory"`, `"orders"`, `"picking"` or `"history"`.

`chartPeriod` renderer.js · line 153

The period chosen on the dashboard chart: `"Week"`, `"Month"` or `"Year"`.

`selectedBatchId`

renderer.js · line 154

The id of the batch shown on the Picking page.

`loadState()`

renderer.js · line 156

Reads the saved state from localStorage. If it exists and has inventory, orders and batches, it runs every upgrade: product statuses, locations and batch stops to the free grid, the layout through `normalizeWarehouseLayout`, a valid history list, and then `upgradeState`. The upgraded data is saved straight back and returned. If nothing is saved, or the text cannot be read (the `try` / `catch` covers broken JSON), it creates fresh demo data, saves it and returns it.

Called from: `state (startup)`

`saveState()`

renderer.js · line 170

Writes the whole state to localStorage as JSON, then refreshes the Orders badge. Every action that changes data ends by calling it.

Called from: `openProductModal`, `openReceiveModal`, `openLocationsModal`, `deleteProduct`, `downloadInventoryTemplate`, `importInventoryCsv`, `openOrderModal`, `generateSampleOrders`, `packOrder`, `shipOrder`, `cancelOrder`, `deleteOrder`, `createAutomaticBatches`, `renderPicking` and 8 more

`recordHistory(action, detail, area, iconName = "history")`

renderer.js · line 171

Adds an entry to the Activity History (action title, detail text, area such as Orders or Picking, an icon name and the time) at the front of `state.history`. Only the newest 150 entries are kept.

Called from: `openProductModal`, `openReceiveModal`, `openLocationsModal`, `deleteProduct`, `downloadInventoryTemplate`, `importInventoryCsv`, `openOrderModal`, `generateSampleOrders`, `packOrder`, `shipOrder`, `cancelOrder`, `deleteOrder`, `createAutomaticBatches`, `renderPicking` and 6 more

`updateReadyBadge()`

renderer.js · line 172

Sets the number on the Orders button in the sidebar to the count of orders that are pending (`status === "ready"`).

Called from: `saveState`, `render`, `startup code`

`escapeHtml(value)`

renderer.js · line 173

Makes text safe to put inside HTML by replacing `&`, `<`, `>`, `'` and `"` with HTML entities. Every name typed by a user (product names, customers, references) goes through it before it is placed in `innerHTML`. Without it, a product called `<b>Box</b>` would be shown in bold, or worse, could inject code.

Called from: `filterChips`, `kpi`, `alertMarkup`, `renderInventory`, `productRow`, `openProductModal`, `openReceiveModal`, `openLocationsModal`, `openStockHistoryModal`, `updateLocationPreview`, `orderActionsMarkup`, `orderRow`, `openOrderModal`, `openOrderDetails` and 5 more

`formatDate(value)`

renderer.js · line 174

Formats a date as month and day, for example "Oct 18", using the browser's `Intl.DateTimeFormat`.

Called from: `orderRow`, `openOrderDetails`

`formatTime(value)`

renderer.js · line 175

Formats a time as hours and minutes, for example "3:45 PM".

Called from: `orderRow`, `openOrderDetails`

`toast(message)`

renderer.js · line 176

Shows a short message in the bottom-right corner for 2.4 seconds. The timer is stored on the function itself (`toast.timer`) so that a new message restarts the countdown instead of being hidden early by the previous one.

Called from: `openProductModal`, `openReceiveModal`, `deleteProduct`, `downloadInventoryTemplate`, `importInventoryCsv`, `openOrderModal`, `generateSampleOrders`, `packOrder`, `shipOrder`, `cancelOrder`, `deleteOrder`, `createAutomaticBatches`, `renderPicking`, `toggleStopPicked` and 6 more

Shared definitions and status rules

`BASE_CATEGORIES`

renderer.js · line 179

The categories offered when adding a product: Food, Snacks, Beverages, Packaging, Equipment, Labels, Safety, General.

`HOLD_STATUSES`

renderer.js · line 180

`["Damaged", "Quarantined"]`: product statuses that block picking.

`localDateKey(value = new Date())`

renderer.js · line 181

A date as `"YYYY-MM-DD"` in the computer's local time zone (not UTC). It is used to group orders due on the same day and to make batch codes.

Called from: `toLocalInputValue`, `createAutomaticBatches`

`isLowStock(item)`

renderer.js · line 182

True when a product still has stock but no more than its reorder point (`0 < quantity <= reorderPoint`).

Called from: `productDisplayStatus`, `renderDashboard`, `buildAlerts`, `renderInventory`, `productRow`

`isOutOfStock(item)`

renderer.js · line 183

True when the quantity is 0 or the status says Out Of Stock.

Called from: `productDisplayStatus`, `renderInventory`, `productRow`

`productDisplayStatus(item)`

renderer.js · line 184

The status shown for a product. A hold status (Damaged, Quarantined) wins first, then Out Of Stock, then Low Stock, otherwise the stored status (normally Available).

Called from: `productRow`

`isOrderDelayed(order)`

renderer.js · line 185

True when an order is still open and its due time has passed.

Called from: `orderDisplayStatus`, `renderDashboard`, `buildAlerts`

`orderDisplayStatus(order)`

renderer.js · line 186

The status shown for an order. Closed orders show their stage (Completed or Cancelled), open late orders show Delayed, and other open orders show their stage.

Called from: `renderDashboard`, `renderOrders`, `orderRow`, `openOrderDetails`

`openBatches()` renderer.js · line 187

All batches that are not complete. Reservations, workload and alerts only count these.

Called from: `reservedBySku`, `reservedByLocation`, `buildAlerts`, `openProductModal`, `deleteProduct`, `availableStockBySku`, `createAutomaticBatches`

Navigation and table filters

Two top-level listeners are set up here. A `keydown` listener (line 189) closes the open modal, or the open row menus, when **Escape** is pressed. A `click` listener on every sidebar button (line 195) calls `navigate` with the button's `data-view`.

`viewFilters` renderer.js · line 196

Remembers the chosen filter chip on the Orders and Inventory pages (both start at "All"), so a filter stays selected when you come back to the page or jump there from the dashboard.

`pendingHighlight` renderer.js · line 197

A CSS selector for the row to highlight after the next page is drawn (set by `navigate`, used by `applyHighlight`).

`navigate(view, options = {})` renderer.js · line 198

Switches to a page. It stores the page in `currentView`, and optionally a filter (`options.filter`), a batch to show (`options.batchId`) and a row to highlight (`options.highlight`). It marks the matching sidebar button as active, redraws with `render()`, moves keyboard focus to the main area without scrolling, and highlights the requested row. Clicking a dashboard card or notification works through this function.

Called from: `startup code`, `renderDashboard`, `openOrderModal`, `generateSampleOrders`, `createAutomaticBatches`, `resetDemoData`

`applyHighlight()` renderer.js · line 208

Finds the element named by `pendingHighlight` and makes it flash. It un-hides the row (it might be filtered out), removes the `flash` class, reads `element.offsetWidth` to force the browser to restart the CSS animation, adds `flash` again, and scrolls the row to the middle of the screen.

Called from: `navigate`

`filterChips(view, options)` renderer.js · line 215

Builds the row of filter buttons ("chips") above a table. Each option has a key, an optional label and an optional count shown in bold. The chip matching `viewFilters[view]` is marked active, with `aria-pressed` for screen readers.

Called from: `renderInventory`, `renderOrders`

`setupTableFilter(view, searchId, rowSelector, emptySelector)` renderer.js · line 219

Makes a table searchable and filterable without redrawing it. Each table row carries `data-search` (all its searchable words in lowercase) and `data-keys` (the chips it belongs to, separated by `|`). The inner function `apply` hides every row that does not contain the search text or does not belong to the chosen chip, and shows the "No … Found" message when nothing is left. Typing calls `apply` again. Clicking a chip stores it in `viewFilters`, updates the chips' active state and calls `apply`.

`apply()` helper inside the function above

Reads the search box and the chosen chip, hides or shows each row, counts the visible rows and toggles the empty message.

Called from: `renderInventory`, `renderOrders`

`render()` renderer.js · line 233

Draws the current page: it refreshes the Orders badge, then calls `renderInventory`, `renderOrders`, `renderPicking`, `renderHistory` or, by default, `renderDashboard`.

Called from: `navigate`, `refreshBehindModal`, `rerenderOrderViews`, `startup code`

`pageHeader(title, description, action = "")` renderer.js · line 242

The HTML for a page's heading: the title, an optional description line and optional buttons on the right.

Called from: `renderDashboard`, `renderInventory`, `renderOrders`, `renderPicking`, `renderHistory`

Part 4B · renderer.js: Dashboard, Inventory and Orders

Every page follows the same pattern, which is worth explaining once in class:

1. **Compute** the numbers the page needs from `state`.
2. **Build** the whole page as one HTML string (a JavaScript template literal with `${…}` placeholders) and assign it to `app.innerHTML`.
3. **Wire** the new buttons: `document.querySelectorAll(...)` finds them and `addEventListener("click", …)` attaches what they do.
4. When a button changes data, it updates `state`, calls `saveState()`, records history, and draws the page again.

Dashboard

`renderDashboard()` renderer.js · line 246

Draws the Dashboard.

- **Counting:** the inner helper `countStatus(label)` counts orders by their *displayed* status (`orderDisplayStatus`), giving Completed, Picking, Pending, and Packing (Picked + Packed). Delayed orders are counted separately (they show as Delayed instead of their stage). It also counts open orders, all non-cancelled orders, and low-stock products, and gets the notifications from `buildAlerts()`.
- **Donut chart:** the status breakdown circle is made with a CSS `conic-gradient`. Each status gets a slice from its start percentage to its end percentage; the variable `cursor` keeps a running total so the slices follow each other.
- **HTML:** four KPI cards (`kpi(...)`), the Order Volume card with Week/Month/Year tabs and an empty `<svg id="order-chart">`, the status list with progress bars, and the notifications list (`alertMarkup` for each alert, or "No Urgent Alerts").
- **Wiring:** the period tabs change `chartPeriod` and redraw; every element with `data-go-view` (cards and status lines) calls `navigate` with its filter; every notification navigates to its target. Then `drawChart(...)` fills the chart, and `syncDashboardPanels` runs on the next frame, once the browser has laid out the page.

`countStatus(label)` helper inside the function above

Counts orders whose displayed status equals `label`.

Called from: `render`

`syncDashboardPanels()` renderer.js · line 294

Makes the Notifications card exactly as tall as the left column (chart + status breakdown), so the two columns line up. On narrow windows (1120 px or less) the columns stack, and the card gets a fixed height of 420 px. It is also called when the window is resized.

Called from: `renderDashboard`, `startup code`

`kpi(tone, iconName, value, label, view, filter)` renderer.js · line 301

The HTML for one KPI card: a coloured icon, the big number and its label. The card is a button carrying `data-go-view` and `data-go-filter`, so clicking it opens Orders or Inventory with the matching filter (for example Low-Stock Products opens Inventory filtered to "Low stock").

Called from: `renderDashboard`

`distribute(total, count)`

renderer.js · line 302

Spreads a total over `count` buckets for the chart, putting each unit in bucket `(i × 3 + 1) % count`. This gives a fixed, even-looking spread. It does **not** use real dates.

> **Watch out.** The Order Volume chart is illustrative. It spreads the *current* totals (all orders, completed, delayed) over the days, weeks or months with `distribute`; it does not read when orders were actually received. Mention this honestly if someone asks how the chart is calculated.

Called from: `drawChart`

`drawChart(period, completed, delayed)`

renderer.js · line 304

Draws the line chart into the SVG (760 × 250 units).

- **Labels:** Mon–Sun for Week, W1–W4 for Month, Jan–Dec for Year.
- **Series:** Received (all orders), Completed and Delayed, each spread with `distribute`.
- **Scales:** the inner functions `x(index)` and `y(value)` turn a label position and a value into SVG coordinates inside margins (left 48, right 18, top 18, bottom 34). The maximum of the y-axis is at least 4.
- **Grid:** five horizontal lines with their values, and the labels along the bottom.
- **Curves:** `smoothPath(points)` builds a smooth SVG path with cubic Bézier curves (`C`). Each segment's control points are pulled 42% of the horizontal distance toward the neighbouring point, which gives soft curves without overshooting.
- **Fill:** the Received line gets a fading area underneath, using a linear gradient.
- **Tooltip:** each data point is a circle. Hovering shows a tooltip ("Tue · Received, 1 order") positioned as a percentage of the chart size, and enlarges the dot.

`x(index)` helper inside the function above

Horizontal position of the label at `index`.

`y(value)` helper inside the function above

Vertical position of a value (higher values are nearer the top, because SVG's y-axis points down).

`smoothPath(points)` helper inside the function above

Turns a list of points into a smooth path: a move (`M`) to the first point, then one cubic curve per following point.

Called from: `renderDashboard`

`buildAlerts()`

renderer.js · line 338

Builds the notification list by checking rules in this order. Each alert has a title, a detail sentence, a severity, an icon and a `go` target, which says where clicking takes you and which row to highlight.

Rule	Severity
A product has zero stock	critical
An open order is past its due time	critical
An urgent order is still pending (not batched)	critical

Rule	Severity
An order is held for stock (from `heldOrders`, with the reasons)	warning
A product is at or below its reorder point	warning
A product is marked Damaged	critical
An open batch is more than 90 minutes old	warning
A product's location is outside the current layout	info
A product's location has no walkway from the start	critical
A location holds more than its capacity	warning
Two products share the same name	warning
Picked orders waiting to be packed / packed orders waiting to ship	info

At the end the list is sorted by severity (critical, then warning, then info). The inner helpers `toProduct(item)` and `toOrder(order)` build the `go` targets, for example `{ view: "inventory", highlight: '[data-product-id="3"]' }`.

`toProduct(item)` helper inside the function above

The navigation target for a product alert: Inventory, filter "All", highlight that product's row.

`toOrder(order)` helper inside the function above

The navigation target for an order alert: Orders, filter "All", highlight that order's row.

Called from: `renderDashboard`

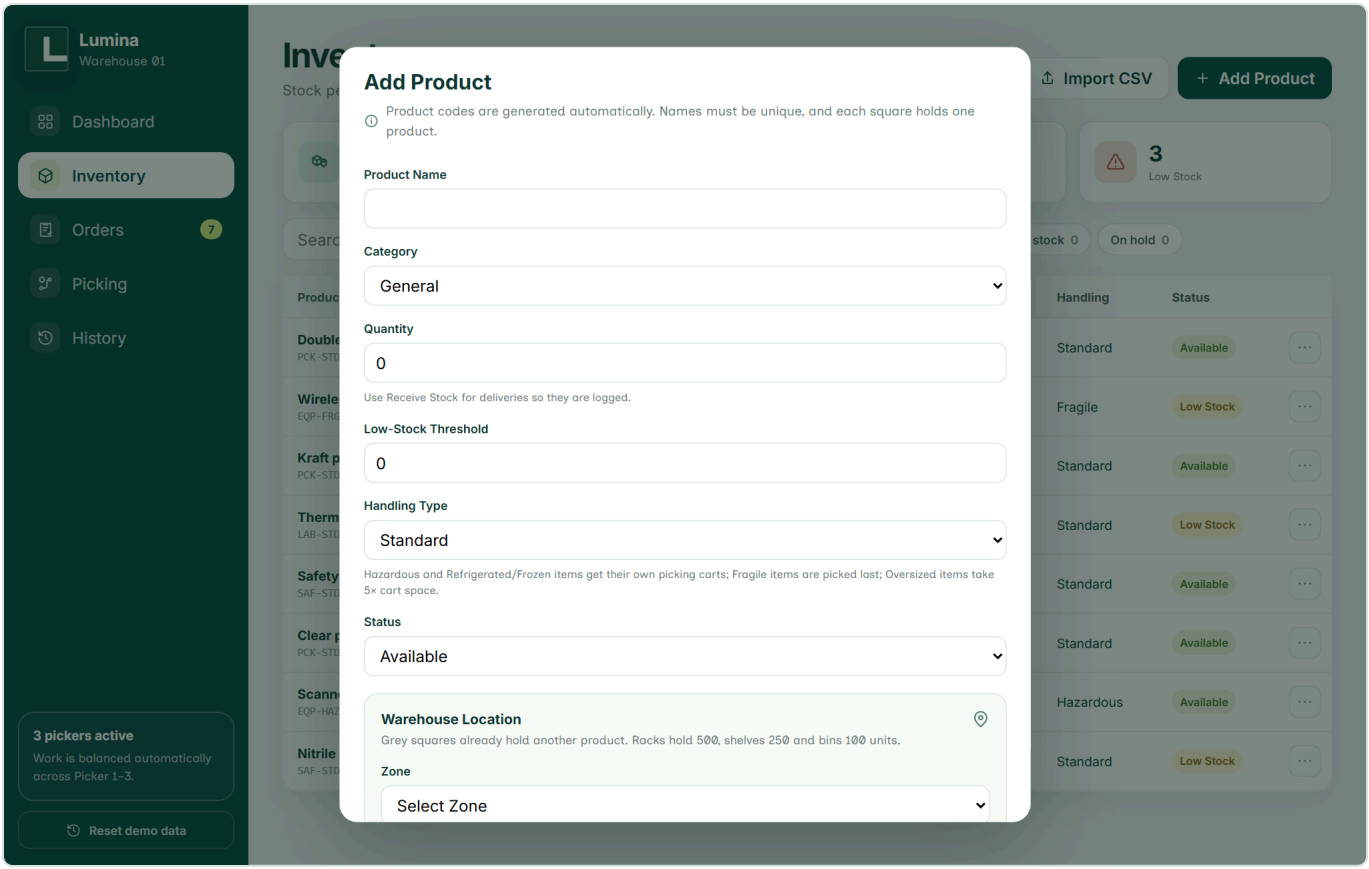
`alertMarkup(alert, index)`

renderer.js · line 362

The HTML for one notification: an icon, the bold title, the detail text and an arrow. The button carries `data-alert="index"`, the alert's position in the list, so the click handler can find its `go` target.

Called from: `renderDashboard`

Inventory



The Add Product window. Clicking a free square on the small map fills in the zone, column, row and container type.

`renderInventory(query = "")`

renderer.js · line 364

Draws the Inventory page. It computes the reserved units per product (`reservedBySku`), the total units, reserved units, low-stock and out-of-stock counts. The header has three buttons (Template, Import CSV, Add Product) and a hidden file input used by Import CSV. There are four summary cards, a search box (pre-filled with `query`, so the search text survives a redraw) and filter chips (All, Low stock, Out of stock, On hold), then a table with one `productRow` per product.

Wiring: `setupTableFilter` makes the table searchable. The header buttons open the product window, download the template, or click the hidden file input. Each row's "···" button opens its menu and closes the others (`stopPropagation` stops the same click from closing it immediately). The inner `bind` helper attaches each menu action to its function: Edit, Receive Stock, Locations & Transfers, Stock History, Delete. Clicking anywhere outside a menu closes all menus.

`bind(attr, fn)` helper inside the function above

For every button with attribute `attr` (for example `data-edit`), on click: close the menus and call `fn` with the number stored in that attribute (the product id).

Called from: `render`, `openProductModal`, `refreshBehindModal`, `deleteProduct`, `importInventoryCsv`

`productRow(item, reserved = 0)`

renderer.js · line 387

The HTML for one table row: name and SKU, category, on hand, reserved, available (on hand − reserved, in red when 0 or below), the primary location plus "+N" when there are more locations (with a tooltip listing them all), handling type, a coloured status pill (good, warn or bad), and the row menu. The row carries `data-search`

with all searchable words and `data-keys` with its chips (Low stock, Out of stock, On hold) for `setupTableFilter`.

Called from: `renderInventory`

`nameTaken(name, exceptSku = null)` renderer.js · line 398

True if another product already has this name (ignoring upper/lower case and surrounding spaces). Product names must be unique because the CSV import matches products by name.

Called from: `openProductModal`

`locationProblem(location, sku, quantity, { allowOwn = [] } = {})` renderer.js · line 400

Checks whether a product may use a square. It returns an error sentence, or an empty string when the square is fine. The checks, in order:

1. the square must be a Rack, Shelf or Bin inside the layout (`isLocationWithinLayout`);
2. no other product may be stored there (`locationOwners`);
3. it must not already be one of this product's own locations, except the ones listed in `allowOwn`;
4. the quantity must fit in the square (`capacityOf`).

Called from: `openProductModal`

`openProductModal(itemId = null)` renderer.js · line 407

Opens the Add Product window, or Edit Product when an id is given.

- **The form:** name, category (the base categories plus the product's own category if it is unusual), quantity (for multi-location products, the quantity at the primary location, with a hint about the total), low-stock threshold, handling type (with an explanation of what each type does), status, and the location: zone, grid column, container type and grid row, plus a live map preview.
- **The map:** the inner function `refreshPreview` runs on every change. It sets the column and row fields' maximum to the zone's size, sets the container type to whatever is on that square, and redraws the preview with `updateLocationPreview`.
- **Saving:** the submit handler checks the name (required and unique), whole numbers for quantity and threshold (`parseWholeNumber`), and the location (`locationProblem`, allowing the product's current primary location). All errors are shown together. A new product gets an automatic SKU (`nextProductId`). The primary location and quantity are updated, a product marked Out Of Stock becomes Available again when it gets stock, and `syncProduct` recalculates the totals.
- **Logging:** a new product records "Initial stock". For an edit, a changed location records two "Moved" movements (out of the old square, into the new one), and a changed quantity records an "Adjustment". If the primary location moved, open picking batches are updated to point to the new square, so nobody is sent to an empty shelf.
- Finally it records history, saves, closes the window, redraws Inventory and shows a toast.

`refreshPreview()` helper inside the function above

Updates the column and row limits and the container type from the layout, then redraws the map preview.

Called from: `renderInventory`

`refreshBehindModal()`

renderer.js · line 475

Redraws the page behind an open window: Inventory keeps its current search text, any other page is simply redrawn.

Called from: `openReceiveModal`, `openLocationsModal`

`locationOptionLabel(entry)`

renderer.js · line 476

The text of a location in a drop-down list, for example `A-01-R01 · 428 / 500 units`.

Called from: `openReceiveModal`, `openLocationsModal`

`openReceiveModal(itemId)`

renderer.js · line 477

The Receive Stock window, for logging a delivery: choose the location to put it away, the number of units and an optional reference (such as `PO-2041`). On submit it checks for a whole number of at least 1 and enough room left in that square. If the delivery is too big, it suggests putting the rest elsewhere through Locations & Transfers. It adds the units, makes an Out Of Stock product Available, records a "Received" movement and history, saves and redraws.

Called from: `renderInventory`

`wireModal(formId, onSubmit)`

renderer.js · line 496

Shared wiring for small windows, so each one needs only its own submit logic. It finds the form and its error box; the inner `fail(message)` shows an error. Typing hides the error. Buttons with `data-close` close the window, and so does a click on the dark background (but not on the window itself). Submitting prevents the browser's page reload and calls `onSubmit(form, fail)`. It also focuses the first input and returns the form.

`fail(message)` helper inside the function above

Shows `message` in the window's error box.

Called from: `openReceiveModal`, `openLocationsModal`, `openStockHistoryModal`, `openOrderDetails`, `openShortPickModal`

`openLocationsModal(itemId, notice = "")`

renderer.js · line 506

The Locations & Transfers window for one product. A table lists each location with its type, units / capacity and units reserved for picking. Three actions are available:

- **Make Primary:** moves that location to the front of the list (picking takes from the primary location first).
- **Transfer:** only shown when the product has more than one location and this location has unreserved units. It opens a small panel to move units to another of the product's locations. The amount must be between 1 and the free (unreserved) units, and must fit in the destination. It records two "Transfer" movements.
- **Remove:** only for an empty, unreserved location, when the product has more than one.

Below the table, **Add A Location** shows the map. The inner `drawPicker(selected)` draws it with occupied squares greyed out. Clicking a free square (or pressing Enter or Space on it) selects it, and the "Add" button adds it as a new empty location. After every change the inner `save(message, history)` re-syncs the product, records history, saves, refreshes the page behind, and reopens the window with a short confirmation.

`save(message, history)` helper inside the function above

Re-syncs the product, records the history entry, saves, redraws the page behind and reopens the window with `message`.

`drawPicker(selected)` helper inside the function above

Draws the location map for choosing a new square, and the Add button when the chosen square is free.

`choose()` helper inside the function above

Selects the clicked square by redrawing the picker with it.

Called from: `renderInventory`

`openStockHistoryModal(itemId)` renderer.js · line 547

The Stock History window: a table of every movement of this product (from `state.movements`) with time, movement type, location, change (green for +, red for −), balance after the change and reference. The inner `when(value)` formats a timestamp as month, day and time.

`when(value)` helper inside the function above

Formats a timestamp as "Oct 18, 3:45 PM". A function with the same name in `openOrderDetails` formats the due date and time.

Called from: `renderInventory`

`closeRowMenus()` renderer.js · line 554

Closes every open "···" row menu.

Called from: `renderInventory`, `bindOrderActions`

`closeModal()` renderer.js · line 555

Closes the open window by emptying `#modal-root`.

Called from: `startup code`, `openProductModal`, `openReceiveModal`, `wireModal`, `openStockHistoryModal`, `openOrderModal`, `rerenderOrderViews`, `openShortPickModal`, `openWarehouseLayoutModal`, `resetDemoData`

`deleteProduct(id)` renderer.js · line 556

Deletes a product, with safety checks. A product that is in an open picking batch cannot be deleted; an alert names the batches to complete first. If pending orders use the product, the confirmation warns that they will be held. After confirmation it removes the product, records history, saves and redraws.

Called from: `renderInventory`

`csvValue(value)` renderer.js · line 564

Prepares one value for a CSV file. A value containing a comma, a quote or a line break is wrapped in quotes, and inner quotes are doubled (the CSV rule).

Called from: `downloadInventoryTemplate`

`downloadInventoryTemplate()` renderer.js · line 565

Creates and downloads `Lumina Inventory Import Template.csv`: a header row with the nine expected columns and one sample row. A byte-order mark at the start makes Excel open it as UTF-8. The file is made in memory as a `Blob`, given a temporary URL, downloaded by clicking an invisible link, and the URL is released afterwards.

Called from: `renderInventory`

`parseCsv(text)`

renderer.js · line 566

Reads CSV text into rows of cells, one character at a time, with a flag that remembers whether it is inside quotes. Inside quotes, `""` is a literal quote and commas or line breaks are part of the value. Outside quotes, a comma ends a cell and a line break (LF or CRLF) ends a row. Completely empty rows are skipped. This handles everything Excel produces, including product names with commas.

Called from: `importInventoryCsv`

`decodeCsvBytes(buffer)`

renderer.js · line 567

Turns the file's bytes into text. It first tries strict UTF-8; if the bytes are not valid UTF-8 (Excel's "CSV" format on Windows saves in the Windows-1252 code page), it decodes them as Windows-1252, so accented letters and symbols still come out right.

Called from: `importInventoryCsv`

`parseWholeNumber(text)`

renderer.js · line 568

Returns the number if the text consists only of digits, otherwise `NaN` (not a number). This rejects "12.5", "-3" and "12 units", which `Number(...)` alone would not all reject.

Called from: `openProductModal`, `openReceiveModal`, `openLocationsModal`, `importInventoryCsv`, `openShortPickModal`

`importInventoryCsv(event)`

renderer.js · line 569

Imports products from a CSV file chosen with the hidden file input.

1. Reads the file as bytes (`FileReader.readAsArrayBuffer`), decodes it and parses it.
2. Normalizes the header names, so "Low-Stock Threshold", "low stock threshold" and "LowStockThreshold" all match (the inner `normalize` keeps only lowercase letters and digits), and checks that all nine required columns exist.
3. Checks each row and collects every problem: product name present and not repeated in the file; quantity and threshold whole numbers; handling type and status recognized (also the old words active / hold / out of stock); the zone exists; column and row are whole numbers; the square is a container, not a walkway.
4. Matches existing products **by name**. An existing product keeps its other locations, and its primary location gets the file's quantity minus what the other locations hold. It checks that the square is free and the quantity fits.
5. Creates or updates each valid product, with "Initial stock", "Moved" or "Adjustment" movements tagged "CSV import".
6. Saves, shows how many products were imported, and lists the skipped rows with reasons (the first 12) in an alert.

`normalize(value)` helper inside the function above

Lowercases a header name and removes everything except letters and digits.

`at(row, name)` helper inside the function above

The trimmed value of the column called `name` in a row.

Called from: `renderInventory`

`nextId(items)`

renderer.js · line 623

The next free id: the largest existing id plus 1.

Called from: `openProductModal`, `importInventoryCsv`, `addOrder`, `createAutomaticBatches`, `engineRouteSvg`, `warehouseLayoutSvg`

`nextProductId(category, handlingType)` renderer.js · line 624

Creates a product code such as `PCK-STD-004`: three letters for the category (Packaging = PCK), three for the handling type (Standard = STD, Fragile = FRG, Refrigerated = CHL, Frozen = FRZ, Hazardous = HAZ, Oversized = OVR), and the next free three-digit number for that prefix.

Called from: `openProductModal`, `importInventoryCsv`

`parseLocation(value = "")` renderer.js · line 625

Splits a location code into its parts `{ zone, aisle, containerType, containerNumber }`, using a regular expression. It also understands the legacy format `A-01-R01-S01-B01`. Anything else gives empty parts.

Called from: `migrateLocationToTwoSides`, `migrateLocationToFreeGrid`, `containerTypeOf`, `openProductModal`, `isLocationWithinLayout`, `bindLocationMap`, `isContainerReachable`, `buildEfficientRoutePlan`, `buildRouteRequest`, `engineRouteSvg`, `warehouseLayoutSvg`, `openWarehouseLayoutModal`

`composeLocation(data)` renderer.js · line 632

The opposite of `parseLocation`: builds a code like `A-03-R04` from the parts, padding numbers to two digits and using R, S or B for the container type.

Called from: `migrateLocationToTwoSides`, `migrateLocationToFreeGrid`, `openProductModal`, `importInventoryCsv`, `updateLocationPreview`, `warehouseLayoutSvg`, `openWarehouseLayoutModal`

`isLocationWithinLayout(value, layout = state.warehouseLayout)` renderer.js · line 633

True when a location code points to a real container: the zone is one of the active zones, the column and row are inside that zone's grid, and the square there has the same container type as the code says.

Called from: `buildAlerts`, `locationProblem`, `openLocationsModal`, `usableLocation`, `generateSampleOrders`, `openWarehouseLayoutModal`

`bindLocationMap(preview, form)` renderer.js · line 634

Makes the squares of a location map clickable: clicking a square (or Enter or Space) copies its zone, column, type and row into the form fields and refreshes the preview.

Called from: `updateLocationPreview`

`updateLocationPreview(form)` renderer.js · line 635

Redraws the location preview in the product window. It reads the form, looks up what is on the chosen square and adopts its container type. It shows a message for the situation: choose a square, "That square is a walkway", "That square is empty", "Already holds …", or "Selected Rack · holds up to 500 units". It then draws the compact map with occupied squares grey, and binds the squares again.

Called from: `openProductModal`, `bindLocationMap`

Stock availability and held orders

`availableStockBySku()` renderer.js · line 643

On-hand quantity minus what open batches will take, per product.

Called from: `freeStockForNewOrders`, `openOrderDetails`

`usableLocation(location)`

renderer.js · line 649

True when a location is inside the layout **and** reachable from the starting point. Only usable locations can be picked from.

Called from: `pickableStockBySku`, `orderBlockers`, `allocateStops`

`pickableStockBySku(exceptBatchId = null)`

renderer.js · line 651

Units a picker can actually take, per product: for each *usable* location, the units there minus the units already reserved there, never below 0. It can ignore one batch's reservations.

Called from: `heldOrders`, `createAutomaticBatches`

`orderBlockers(order, available)`

renderer.js · line 656

Lists why an order cannot be batched yet. It first adds up the quantity needed per product (an order could list the same product twice). Then for each product: no longer in inventory, on hold or out of stock, no reachable location, or "short by N" when `available` has too little. An empty list means the order can go.

Called from: `heldOrders`, `createAutomaticBatches`

`heldOrders()`

renderer.js · line 668

Works out which pending orders are held, and why. It goes through pending orders in priority order (`sortReadyOrders`). An order without problems "takes" its units from the available stock, so the next orders see less. This is exactly how batching will allocate stock, so urgent orders get the stock first. Returns a `Map` from order id to its list of problems.

Called from: `buildAlerts`, `renderOrders`, `openOrderDetails`

`sortReadyOrders(orders)`

renderer.js · line 673

Sorts orders by priority (urgent, then express, then standard) and, for equal priority, by earliest due time. It returns a new array and does not change the original.

Called from: `heldOrders`, `createAutomaticBatches`, `completeBatch`

Orders

`renderOrders()`

renderer.js · line 675

Draws the Orders page. The header has **Add sample orders**, **New order** and **Create automatic batches**; the last is disabled unless there are more pending orders than held ones, so at least one can be batched. The filter chips with counts are All, Open, Pending, Picking, To pack, To ship, Delayed, Held, Completed, Cancelled (the inner helpers `countOf` and `countStatus` count by displayed and stored status). The table shows the newest orders first. Wiring connects the search, the three header buttons, the row actions (`bindOrderActions`) and closing menus on outside clicks.

`countOf(label)` helper inside the function above

Counts orders by their displayed status.

Called from: `render`

`bindOrderActions(root)`

renderer.js · line 694

Connects the order buttons inside `root`. It is used by both the Orders table and the order details window, so the buttons behave the same in both places. The "···" menus open and close, and the inner `bind` connects View Details, Edit, Cancel, Delete, Pack and Ship to their functions.

Called from: `renderOrders`, `openOrderDetails`

`orderActionsMarkup(order, { inline = true } = {})`

renderer.js · line 699

Decides which actions an order offers, based on its status:

- the **next step** button: **Pack** when picked, **Ship** when packed;
- **View Details** always;
- **Edit** only while pending;
- **Cancel** while pending or batched;
- **Delete** while pending, cancelled or shipped.

With `inline: false` it returns the pieces (`{ next, items }`) for the details window instead of the table's HTML.

Called from: `orderRow`, `openOrderDetails`

`orderRow(order, problems = [])`

renderer.js · line 708

The HTML for one order row: the order number (a link to the details), customer, priority pill (red for urgent, amber for express), due date and time, units and number of products (with a tooltip listing them), status pill with extra notes, the batch cell, and the actions. The **batch cell** shows the batch code; or a red **Held** pill with the first reason ("+N more" if there are several); or "Awaiting" for a pending order that can be batched; or "—". The **status note** explains a Delayed order's real stage, or that an order went back to pending after a short pick.

Called from: `renderOrders`

`nextOrderNumber()`

renderer.js · line 721

The next order number: the highest number at the end of any existing order number (at least 10480) plus 1, as `SO-10488` and so on.

Called from: `addOrder`

`toLocalInputValue(date)`

renderer.js · line 722

Formats a date for an `<input type="datetime-local">` field (`YYYY-MM-DDTHH:MM` in local time).

Called from: `openOrderModal`

`freeStockForNewOrders()`

renderer.js · line 724

Stock a new order could still use: on hand, minus open batches, minus what pending orders are already waiting for. The new-order window uses it to warn early when an item will be held.

Called from: `openOrderModal`, `generateSampleOrders`

`addOrder({ customer, priority, dueAt, lines })`

renderer.js · line 725

Creates a pending order with the next id and order number and `events.createdAt`, adds it to `state.orders` and returns it. Saving is left to the caller.

Called from: `openOrderModal`, `generateSampleOrders`

`openOrderModal(orderId = null)`

renderer.js · line 731

The New order window, or Edit for a pending order.

- Only pending orders can be edited, and there must be at least one product.
- The default due time is 4 hours from now, rounded up to the next quarter hour.
- **Items** are added as rows (product drop-down, quantity, remove button). The inner `addLine` picks a sensible default product: one not yet used, not on hold, and with free stock.
- The inner `refreshHints` adds up the quantity per product across all rows and shows next to each row "N available", "Short by N: will be held", or "Quarantined: will be held". When editing, the order's own units count as available.
- **Saving** checks the customer, the due date, and quantities (whole numbers of at least 1, at least one item). Duplicate products are merged into one line. It creates the order (`addOrder`) or updates it (clearing the short-pick note), records history, saves, and jumps to the Orders page with the order highlighted.

`refreshHints()` helper inside the function above

Recomputes the availability hint of every item row, and disables the remove button when only one row is left.

`addLine(presetSku, presetQuantity)` helper inside the function above

Adds an item row, preset to a product and quantity when editing, and connects its events.

Called from: `renderOrders`, `bindOrderActions`

`SAMPLE_CUSTOMERS`

renderer.js · line 796

Twelve made-up customer names used by **Add sample orders**.

`generateSampleOrders(count = 3)`

renderer.js · line 797

Creates random demo orders. It only uses products that can really be picked: not on hold, in stock, inside the layout and reachable. Each order gets a random priority (15% urgent, 35% express, 50% standard), a due time 1–8 hours ahead on a quarter hour, 1–3 different products that still have free stock, and quantities of 1–12 kept below a third of the spare stock. Free stock is reduced as it goes, so the sample orders do not all compete for the same units. Then it records history, saves, and shows the Orders page with the last new order highlighted.

`random(n)` helper inside the function above

A random whole number from 0 to n − 1.

Called from: `renderOrders`

`rerenderOrderViews()`

renderer.js · line 814

After an order action, closes the details window if it is open and redraws the current page.

Called from: `packOrder`, `shipOrder`, `cancelOrder`, `deleteOrder`

`packOrder(id)`

renderer.js · line 815

Moves a picked order to **packed** and records the time (`events.packedAt`) and history.

Called from: `bindOrderActions`

`shipOrder(id)`

renderer.js · line 816

Moves a packed order to **shipped** (shown as Completed) and records the time and history.

Called from: `bindOrderActions`

`cancelOrder(id)`

renderer.js · line 817

Cancels a pending or batched order after confirmation. A batched order is first taken out of its batch (`detachOrderFromBatch`), which recalculates that batch's stops. The order gets status `cancelled` and the cancellation time.

Called from: `bindOrderActions`

`deleteOrder(id)`

renderer.js · line 826

Deletes a pending, cancelled or shipped order after confirmation. Orders in picking or packing cannot be deleted, so no stock is left half-processed.

Called from: `bindOrderActions`

`openOrderDetails(id)`

renderer.js · line 833

The order details window. It shows the status and summary (priority, due date and time, handling class, cart load), a table of the lines with each product's location, available stock (red if too little) and handling type, the hold reasons if the order is held, a note if it went back after a short pick, a **timeline** (Created, Batched, Picked, Packed, Shipped and, if any, Cancelled, each with its time), and the picking information (batch, picker, stops picked). The action buttons come from `orderActionsMarkup` and are connected with `bindOrderActions`.

Called from: `bindOrderActions`, `renderPicking`

`capitalize(value)`

renderer.js · line 851

Makes the first letter uppercase ("express" becomes "Express").

Called from: `orderRow`, `openOrderModal`, `openOrderDetails`

Part 4C · renderer.js: batching and picking

This part turns pending orders into picking batches (carts), lets pickers work through them stop by stop, and updates the stock when a batch is completed.

Turning orders into batches

`allocateStops(orders, exceptBatchId = null)` renderer.js · line 854

Turns orders into **picking stops**, the places the picker must visit and how many units to take at each.

1. Adds up the quantity needed per product over all the orders.
2. For each product, takes units from its **primary location first**, then from its other locations with the most stock first. Only usable (reachable) locations count, and units already reserved by other batches are left alone.
3. Each stop has the id `sku@location`. If the same stop comes up twice, the inner `add` merges them into one stop.
4. If something is still missing (which should not happen, because blocked orders are held earlier), a stop for the remainder is added at the primary location (or "Unassigned"), so the shortage stays visible.
5. Numbers the stops (`sequence` 1, 2, 3 ...).

`add(stop)` helper inside the function above

Adds a stop, or increases the quantity of an existing stop with the same id.

Called from: `createAutomaticBatches`, `detachOrderFromBatch`

`batchHandlingClass(orders)` renderer.js · line 872

The handling class of a group of orders: Hazardous if any order is, otherwise Cold chain if any order is, otherwise Standard.

Called from: `detachOrderFromBatch`

`createAutomaticBatches()` renderer.js · line 873

The **Create automatic batches** button. This is the main business logic of Lumina.

1. **Choose the orders.** Takes the pending orders in priority order (`sortReadyOrders`) and checks each against the pickable stock (`orderBlockers`). Accepted orders take their units from the available stock; the others are listed as held with their reasons. If none can be batched, it explains why and stops.
2. **Group compatible orders.** Orders share a cart only when they have the same handling class, the same priority and the same due day. The group key looks like `"Standard|express|2026-10-18"`. This keeps hazardous and cold-chain goods away from everything else, and keeps urgent work together.
3. **Fill carts.** Within each group, orders are added to the current cart until it would have more than 4 orders or more than 60 load units; then a new cart starts. An order bigger than a whole cart gets a cart of its own.
4. **Balance the pickers.** It measures each picker's unfinished work as the load of their open batches. Each new cart goes to the picker with the least work (ties go to the lower-numbered picker), and that picker's workload grows by the cart's load.
5. **Create each batch.** The code is `B-` + the priority's first letter (U, E or S) + `H` for hazardous or `C` for cold chain + `-` + the date as YYMMDD + `-` + the batch number, for example `B-EH-261018-03` (express, hazardous, October 18, 2026, batch 3). The stops are made with `allocateStops` and put in a good walking

order straight away with the built-in routing (`optimizeRouteStops`). The batch is saved with status `planned`, its orders, route, picker, progress 0, time, handling class and load.

6. **Update the orders.** Each order becomes `batched` with its batch code and the time it was batched.
7. Records history, saves, opens the Picking page, shows a toast and, a moment later, an alert listing the held orders, if any.

> **For your report.** This function is a good one to explain step by step in class: it shows sorting (priorities), filtering (stock checks), grouping (a `Map` of groups), a greedy "fill until full" algorithm (carts) and load balancing (pickers). The screenshot of the Picking page shows its result: four batches, two of them hazardous on their own carts.

Called from: `renderOrders`, `renderPicking`

The Picking page

`isStopPicked(batch, stop)`

renderer.js · line 904

True when the stop is ticked, or when the whole batch is complete.

Called from: `batchPickCounts`, `renderPicking`

`batchPickCounts(batch)`

renderer.js · line 905

How many stops are picked, the total number of stops, and the percentage (rounded). It is used for progress bars and the "N of M stops picked" text.

Called from: `openOrderDetails`, `renderPicking`, `toggleStopPicked`, `openShortPickModal`, `detachOrderFromBatch`

`handlingBadge(handlingClass)`

renderer.js · line 906

A red "Hazardous" or blue "Cold chain" pill for non-standard batches (nothing for Standard).

Called from: `renderPicking`

`renderPicking()`

renderer.js · line 907

Draws the Picking page for the selected batch (or the newest one).

- **No batches:** shows an empty message with a **Create Automatic Batches** button, plus **Edit Warehouse Layout**.
- **Route plan:** `hasValidPlan(selected)` says whether a saved Java plan still matches the batch and layout. If not, the stops are put in built-in order again (`optimizeRouteStops`) and `builtinRoutePlan` computes the walking legs to draw and any unreachable stops.
- **Next stop:** the first stop in route order that is not picked and not unreachable. It is shown in a banner with **Short Pick** and **Mark picked**. When everything is picked, the banner says to walk back to S and complete the batch.
- **Batch list** (left): every batch with its code, handling badge, number of orders and stops, picker and a progress bar.
- **Route card:** the batch code, a picker drop-down (to reassign the batch), orders, progress, cart load out of 60 (noting "single large order" when one big order exceeds the cart), and the engine chip (`engineStatusMarkup`). **Complete Batch** is disabled until every stop is picked and none is unreachable. The map comes from `warehouseLayoutSvg` with the route, the picked stops and the next stop.

- **Stop sequence:** a card per stop (number or check mark, product, location, units, extra state: Unreachable, Short, Picked or Next) with a **Short** button. Fragile stops carry a "Fragile · picked last" tag (the inner `fragileTag`). A final card "Back to the start" shows the return to S.
- **Orders in this batch,** each with a **Remove** button while the batch is open.
- **Wiring:** selecting a batch, Edit Layout, ticking stops, Short, Remove, order details, reassigning the picker (recorded in history), Cancel and Complete.
- Last, if there is no valid plan, it asks the Java engine for one (`requestRoutePlan`). When the answer comes back, the page redraws itself with the optimized route.

`fragileTag(stop)` helper inside the function above

Returns the "Fragile · picked last" tag for fragile stops.

Called from: `render`, `toggleStopPicked`, `openShortPickModal`, `removeOrderFromBatch`, `cancelBatch`, `completeBatch`, `requestRoutePlan`, `openWarehouseLayoutModal`

`toggleStopPicked(batch, id)` renderer.js · line 947

Ticks or unticks a stop. Unticking also clears a short-pick record. It updates the batch's progress and status (`active` once anything is picked, otherwise `planned`), saves and redraws. When the last stop is ticked, the toast reminds the picker to walk back to S and complete the batch.

Called from: `renderPicking`

`openShortPickModal(batch, id)` renderer.js · line 955

The Short Pick window, used when fewer units were on the shelf than expected. The picker enters the units actually found, from 0 to one less than expected (if all were there, they should use Mark picked). The stop is marked picked, `short`, with `pickedQty` set to the units found, and the event is recorded in history. The stock is corrected later, when the batch is completed.

Called from: `renderPicking`

`detachOrderFromBatch(batch, order)` renderer.js · line 971

Takes an order out of an open batch.

- If no orders remain, the batch is deleted and the result is `"emptied"`.
- Otherwise the stops are recalculated for the remaining orders (`allocateStops` with `exceptBatchId`, so this batch's own reservations do not block it). Ticks are kept for stops that still have the same id and quantity. The new route is ordered (`optimizeRouteStops`), the handling class, load and progress are recalculated, and the result is `"updated"`.

Called from: `cancelOrder`, `removeOrderFromBatch`

`removeOrderFromBatch(batch, orderId)` renderer.js · line 982

The **Remove** button on the Picking page. After confirmation, it detaches the order (above), sets it back to pending, records whether the batch was updated or closed, saves and redraws.

Called from: `renderPicking`

`cancelBatch(batch)` renderer.js · line 989

Cancels an open batch after confirmation. All of its orders go back to pending, the batch is deleted, and nothing is taken from stock (nothing was picked for real yet).

Called from: `renderPicking`

`completeBatch(batch)`

renderer.js · line 999

Complete Batch: turns what was picked into real stock changes and decides which orders can go to packing. This is the most careful function in the app.

1. **Take the stock off the shelves.** For each stop, the units really picked are the expected quantity, or the found quantity for a short pick. They go into a `pool` per product, are subtracted from that location (a "Picked" movement with the batch code), and the product totals are re-synced.
2. **Correct short-picked shelves.** A short pick proves the shelf held only what was found. Whatever the system still thought was there is set to 0 with a "Count correction" movement ("Short pick in B-...: found 3 of 5").
3. **Fill the orders, most urgent first.** Going through the batch's orders by priority and due time, an order whose every product is available in the pool takes its units and becomes `picked` (time recorded). Otherwise it goes back to `ready`, is removed from the batch and is flagged `shortPicked`.
4. **Return the leftovers.** Units in the pool that no order could use (they were picked for an order that went back to pending) are put back on the shelf they came from, with a "Returned to stock" movement.
5. Marks the batch complete (100%, completion time, every stop ticked), records history for the stock correction and the completion (orders sent to packing, orders back to pending), saves, redraws and shows a toast.

> **Note.** Why "return to stock"? Suppose two orders both need 4 scanners and the shelf only had 6. The urgent order gets 4 and goes to packing. The other order cannot be filled, so its 2 picked scanners go back on the shelf, and the order waits in Pending for the next batch.

Called from: `renderPicking`

Part 4D · renderer.js: paths, routes, the map and the layout editor

Walking on the grid: breadth-first search

The JavaScript side finds walking paths with **breadth-first search (BFS)**. On a grid where every step costs the same (one square), BFS explores squares in rings of increasing distance from the start, so the first time it reaches a target it has found a shortest path. This is the key algorithm of the built-in routing and of the "is this location reachable?" check.

`gridKey(row, column)`

renderer.js · line 1031

Makes the key `"row-column"` (for example `"8-4"`) used to store squares in a zone's `cells` object.

Called from: `normalizeWarehouseLayout`, `importInventoryCsv`, `updateLocationPreview`, `pathNeighbors`, `adjacentPathKeys`, `boundaryPathKeys`, `engineRouteSvg`, `warehouseLayoutSvg`, `openWarehouseLayoutModal`

`gridPosition(key)`

renderer.js · line 1032

The opposite of `gridKey`: turns `"8-4"` back into `{ row: 8, column: 4 }`.

Called from: `pathNeighbors`, `zoneEntryPath`, `isCorridorEdge`, `layoutCellCenter`, `corridorLinks`, `buildRouteRequest`, `builtinRoutePlan`, `openWarehouseLayoutModal`

`pathNeighbors(zone, key)`

renderer.js · line 1033

The walkable neighbours of a square: the squares above, below, left and right that are inside the zone and are Path squares. Diagonal steps are not allowed, just as a cart cannot cut corners between racks.

Called from: `shortestPathToAny`

`adjacentPathKeys(zone, row, column)`

renderer.js · line 1034

The Path squares next to a storage square, meaning where a picker can stand to pick from it. A rack between two aisles can be picked from either side, so it can have more than one.

Called from: `isContainerReachable`, `buildEfficientRoutePlan`

`boundaryPathKeys(zone, orientation = "Vertical")`

renderer.js · line 1035

The Path squares on the zone's edge that faces the dispatch corridor: the bottom row in a vertical layout, or column 1 in a horizontal one.

Called from: `zoneEntryPath`, `zoneDoorKey`

`zoneEntryPath(zone, orientation = "Vertical")`

renderer.js · line 1036

The default entry square of a zone: the corridor-edge Path square closest to the middle of the edge (or `null` if the edge has none).

Called from: `normalizeWarehouseLayout`, `warehouseStartingPoint`, `zoneDoorKey`, `openWarehouseLayoutModal`

`warehouseStartingPoint(layout)`

renderer.js · line 1037

Where routes start and end: the saved starting point if it is on a Path square, otherwise zone A's entry square (or its first Path square).

Called from: zoneDoorKey, startToCorridorPath, isContainerReachable, buildEfficientRoutePlan, buildRouteRequest, builtinRoutePlan, warehouseLayoutSvg

shortestPathToAny(zone, start, targets)

renderer.js · line 1038

Breadth-first search inside one zone: the shortest walk from `start` to whichever square in the `targets` set is closest. It returns the list of squares from start to goal, or `null` if no walkway connects them.

renderer.js · lines 1038–1038

```
1038  function shortestPathToAny(zone,start,targets) { if (!start || !targets.size) return null; const queue = [start]; const parents = new Map([[start,null]]); let goal = targets.has(start) ? start : null; while (queue.length && !goal) { const current = queue.shift(); for (const next of pathNeighbors(zone,current)) if (!parents.has(next)) { parents.set(next,current); if (targets.has(next)) { goal = next; break; } queue.push(next); } } if (!goal) return null; const path = []; for (let cursor = goal; cursor; cursor = parents.get(cursor)) path.unshift(cursor); return path; }
```

How it works:

- `queue` holds squares still to explore, oldest first (`shift()` takes from the front), which makes the search go outward ring by ring.
- `parents` remembers, for every square seen, the square we came from. This also marks squares as visited, so none is explored twice.
- As soon as a neighbour is in `targets`, the search stops. Because of the ring order, it is a closest target.
- The path is rebuilt backwards: from the goal, follow `parents` to the start, putting each square in front (`unshift`).

Each square is visited at most once, so the search takes time proportional to the number of squares in the zone.

Called from: zoneDoorKey, startToCorridorPath, isContainerReachable, buildEfficientRoutePlan, builtinRoutePlan

isCorridorEdge(zone, key, orientation)

renderer.js · line 1040

True when a square is on the zone's corridor edge (the bottom row, or column 1 in a horizontal layout). Doors must be there.

Called from: zoneDoorKey, openWarehouseLayoutModal

zoneDoorKey(layout, zoneName)

renderer.js · line 1043

The door square through which a zone opens onto the corridor (one door per zone).

1. A door placed in the grid editor wins, if it is still a Path square on the corridor edge.
2. For the zone with the starting point, the door is the corridor-edge square nearest to S (found with `shortestPathToAny`).
3. For other zones, it is the middle walkway square of the edge (`zoneEntryPath`).

Called from: startToCorridorPath, isContainerReachable, buildEfficientRoutePlan, corridorLinks, builtinRoutePlan, corridorLaneSvg, openWarehouseLayoutModal

startToCorridorPath(layout)

renderer.js · line 1051

The walk from the starting point to its zone's door, or `null` when S is cut off from the corridor. If this is `null`, no other zone can be reached.

Called from: isContainerReachable, buildEfficientRoutePlan

`isContainerReachable(value, layoutOverride = null)` renderer.js · line 1052

Can a picker reach this location?

- If the location is in the **same zone as S**, it is reachable when BFS finds a walk from S to a Path square next to it.
- If it is in **another zone**, S must reach the corridor (`startToCorridorPath`), and that zone's door must reach a Path square next to the location.

It accepts another layout, so the layout editor can check a new layout before saving it.

Called from: `buildAlerts`, `usableLocation`, `generateSampleOrders`, `openWarehouseLayoutModal`

`stopIsLast(stop)` renderer.js · line 1060

True when a stop's product is Fragile. Fragile items are picked last so they ride on top of the cart.

Called from: `renderPicking`, `buildEfficientRoutePlan`, `routePlanKey`, `buildRouteRequest`

`stopKey(stop)` renderer.js · line 1061

A stop's unique key: its `id`, or `sku@location` for old data without an id.

Called from: `renderPicking`, `toggleStopPicked`, `openShortPickModal`, `detachOrderFromBatch`, `routePlanKey`, `buildRouteRequest`, `requestRoutePlan`, `builtinRoutePlan`, `engineRouteSvg`

The built-in routing

`buildEfficientRoutePlan(route = [], layoutOverride = null)` renderer.js · line 1063

Lumina's own route planner, used in the browser, whenever Java is unavailable, and as the "baseline" the Java engine compares itself with. It is a **greedy** method: always walk to the nearest remaining stop.

1. Prepares each stop with its original position, its parsed location and whether it is fragile (`_last`).
2. Plans two groups in turn: first the **non-fragile** stops, then the **fragile** ones.
3. Within a group, visits the zones one at a time: the zone the picker is in first, then the others in alphabetical order (reverse order in a horizontal layout); the inner `zoneRank` decides this.
4. Inside a zone it starts where the picker stands, or at the zone's door when coming from the corridor. It repeatedly runs BFS (`shortestPathToAny`) to every remaining stop's access squares and walks to the closest one; ties go to the stop that came first. Every walk is saved as a *segment*.
5. Stops that cannot be reached (no door path, no corridor connection, no walkway next to them, or no valid zone) are put in an `unreachable` list.
6. Returns `ordered` (the reachable stops in visiting order, then the unreachable ones, renumbered and without the helper fields, using the inner `clean`), `segments` (the walks) and `unreachable`.

`zoneRank(name)` helper inside the function above

The sort value of a zone: its letter's character code, made negative for horizontal layouts so the order is reversed.

`clean(stop, index)` helper inside the function above

Removes the temporary helper fields (`_originalIndex`, `_location`, `_last`) and sets the stop's new sequence number.

Called from: `optimizeRouteStops`, `buildRouteRequest`, `builtinRoutePlan`

`optimizeRouteStops(route)` renderer.js · line 1094

A shortcut: the stops of a route in the built-in visiting order (`buildEfficientRoutePlan(route).ordered`).

> **Note.** Greedy "nearest next" is fast and usually good, but not always the best. The Java engine finds the truly shortest round trip for up to 12 stops. The chip on the Picking page shows how many steps the Java route saves compared with the built-in one ("… · 6 fewer than built-in").

Called from: `createAutomaticBatches`, `renderPicking`, `detachOrderFromBatch`, `openWarehouseLayoutModal`

Talking to the Java engine (desktop app)

To compare walking distances across zones, the corridor needs a length in "squares". These functions measure the map in drawing units and convert corridor distances to square-widths, the unit the Java engine uses.

`layoutBlocks(layout)` renderer.js · line 1097

Works out where each zone's rectangle is drawn on the 760-unit-wide map. In a **vertical** layout the zones stand side by side with gaps of 22, 34 or 48 units (Compact, Standard, Wide spacing), each 238 units tall. In a **horizontal** layout they are stacked beside a vertical corridor, with square cells sized between 12 and 28 units so that the grid stays readable.

Called from: `corridorLinks`, `buildRouteRequest`, `warehouseLayoutSvg`

`layoutCellCenter(block, zone, key)` renderer.js · line 1114

The centre point (x, y) of a square on the drawing.

Called from: `corridorLinks`, `corridorLaneSvg`, `engineRouteSvg`, `warehouseLayoutSvg`

`corridorLinks(layout, blocks = layoutBlocks(layout))` renderer.js · line 1116

For each zone, its door square and its position along the corridor: `px` in drawing units, and `pos` in **square-widths** (`px` divided by the average square size, rounded to 3 decimals). The Java engine uses `pos` to know how far apart two doors are: walking from zone A's door to zone B's costs `|pos B − pos A|` steps.

Called from: `buildRouteRequest`, `builtinRoutePlan`, `engineRouteSvg`

`javaEngine` renderer.js · line 1127

The page's view of the engine: `available` is true only if `window.lumina.optimizeRoute` exists (the desktop app); `inflight` holds the plan keys being computed right now; `failedKeys` holds the keys Java could not plan, so they are not retried endlessly; `lastError` keeps the last error message for the chip's tooltip.

`hashText(text)` renderer.js · line 1128

A fast 32-bit **FNV-1a hash** of a text, written in base 36. Each character is mixed in with XOR and a multiplication by the FNV prime 16777619 (`Math.imul` does 32-bit integer multiplication). The same text always gives the same short code, and a small change gives a completely different one.

Called from: `routePlanKey`

`routePlanKey(batch)` renderer.js · line 1129

A fingerprint of "this batch in this warehouse": the hash of the whole layout plus the sorted list of stop keys (with `!` added for fragile stops). If either the layout or the stops change, the key changes and the old plan is no longer valid.

Called from: `hasValidPlan`, `requestRoutePlan`, `engineStatusMarkup`

`hasValidPlan(batch)`

renderer.js · line 1130

True when the batch has a saved Java plan whose key equals the current `routePlanKey`.

Called from: `renderPicking`, `requestRoutePlan`

`buildRouteRequest(batch)`

renderer.js · line 1131

Builds the request sent to Java:

- `zones`: each zone's name, rows, columns and the list of Path squares as `[row, column]` pairs;
- `start`: the starting point's zone, row and column;
- `corridor`: each door square with its `pos` along the corridor (from `corridorLinks`);
- `stops`: each stop's id, zone, row, column and `last` (fragile);
- `baselineOrder`: the built-in visiting order, so Java can report how much shorter its route is;
- `returnToStart: true`, because the route ends back at S.

Part 5 shows a real request built by this function and Java's answer.

Called from: `requestRoutePlan`

`requestRoutePlan(batch)`

renderer.js · line 1143

Asks Java for a plan, without waiting. It stops at once if Java is not available, the batch is complete or empty, the plan is already valid, or the same request is already running or has failed before. Otherwise it records the key as in flight and calls `window.lumina.optimizeRoute(buildRouteRequest(batch))`.

When the answer arrives, the inner `finish` runs:

- On failure it remembers the key as failed and keeps the error message. The chip will say "Java engine unavailable" and the built-in route stays.
- On success, and only if the batch still matches the key (the user may have changed it meanwhile), it reorders the stops in Java's order (stops Java did not return go at the end), saves the plan (distance, baseline distance, method, time taken, legs, unreachable stops) and saves the state.
- If the user is still looking at that batch, it redraws the Picking page.

Any error, thrown or returned, becomes a failed result, so the page never crashes because of Java.

`finish(result)` helper inside the function above

Handles Java's answer (or failure) as described above.

Called from: `renderPicking`

`engineStatusMarkup(batch, planValid, builtinDistance)`

renderer.js · line 1164

The small chip under the batch title:

- with a valid plan: "Java route engine · 85 steps round trip · shortest possible" (or "optimized" for the heuristic), plus "· N fewer than built-in" when Java saved steps. The tooltip shows the engine version and planning time.
- in a browser: "Built-in routing · N steps round trip";
- when Java failed: a warning chip "Built-in routing · Java engine unavailable", with the error as a tooltip;
- while waiting: a pulsing "Java engine is optimizing this route...";

- nothing for a completed batch.

Called from: `renderPicking`

`builtinRoutePlan(route, layout)` renderer.js · line 1176

Converts the built-in plan into the same shape as Java's answer, so the map can draw both the same way. Each **leg** is a list of points: squares `{ zone, row, col }` or corridor points `{ corridor: index }`.

- When the next stop is in another zone, the inner `crossTo` walks out of the current zone to its door, steps onto the corridor at that door, then along the corridor to the next zone's door.
- After the last stop, it adds a **return leg** back to S, crossing zones if needed.
- It also returns the unreachable stops and the total distance (`routeLegsDistance`).

`linkIndex(zoneName)` helper inside the function above

The position of a zone's door in the corridor list.

`square(zone, key)` helper inside the function above

Turns a grid key into a point `{ zone, row, col }`.

`crossTo(from, zoneName, points)` helper inside the function above

Adds the points to walk from a square to its zone's door, onto the corridor, and along the corridor to another zone's door.

Called from: `renderPicking`, `warehouseLayoutSvg`

`routeLegsDistance(legs, links)` renderer.js · line 1206

The walking distance of a route in square-widths, computed exactly like Java does. Each step between two squares counts 1 (0 if it is the same square), a step between a square and the corridor counts 1, and a move along the corridor counts the difference between the two doors' `pos` values.

Called from: `builtinRoutePlan`

Drawing the warehouse map

`corridorLaneSvg(layout, blocks)` renderer.js · line 1216

Draws the dispatch corridor as part of the building: a rounded grey lane below the zones (or beside them in a horizontal layout), a short connector from each zone's door to the lane, and the label "DISPATCH CORRIDOR".

Called from: `warehouseLayoutSvg`

`engineRouteSvg(routePlan, route, layout, blocks, { compact, pickedIds, nextId })` renderer.js · line 1229

Draws a route (Java's or the built-in one) on top of the map.

- The inner `toPoint` converts each point of a leg to drawing coordinates (corridor points go on the corridor line).
- Each leg becomes a dashed dark line; picked legs fade.
- At the end of each leg there is a numbered circle: a check mark and green when picked, yellow with a glow when it is the next stop, white otherwise. A short red line connects the standing square to the rack being picked (the inner `locate` finds the rack's position).
- The return leg is drawn lighter and dotted.

- Unreachable stops get a red "!" marker.

`locate(location)` helper inside the function above

The drawing position of a location code's square.

`toPoint(point)` helper inside the function above

The drawing position of a route point (a square or a corridor door).

Called from: `warehouseLayoutSvg`

`warehouseLayoutSvg({ route, selectedLocation, compact, layoutOverride, interactive, pickedIds, nextId, enginePlan, occupiedBy } = {})` renderer.js · line 1251

The one function that draws every warehouse map in the app: on the Picking page, in the location pickers and in the layout editor preview.

- Every square of every zone is drawn as a rectangle coloured by type: Path white, Rack green, Shelf yellow, Bin blue, empty pale, occupied grey. Containers show their letter (R, S or B).
- The selected square gets a red outline.
- In **interactive** mode, free containers become keyboard-focusable buttons (`role="button"`, `tabindex="0"`) with a tooltip ("Rack A-01-R01 · holds up to 500 units"). Occupied squares get a tooltip naming the product there.
- Each zone gets a border and a "ZONE A" label; then come the corridor (`corridorLaneSvg`), the route layer (the Java plan if valid, otherwise the built-in plan) and the black **S** start marker.

Called from: `openLocationsModal`, `updateLocationPreview`, `renderPicking`, `openWarehouseLayoutModal`

The warehouse grid editor

`openWarehouseLayoutModal()` renderer.js · line 1285

The **Warehouse Grid Editor** window. All editing happens on a `draft` (a deep copy made with `structuredClone`), so nothing changes until **Save**. The set `dirtyZones` remembers which zones were edited.

- **Settings:** number of zones (2–4), orientation (Vertical or Horizontal) and zone spacing.
- **Zone tabs:** a dot marks the edited zones.
- **The palette:** Starting Point, Door, Path, Rack, Shelf, Bin and Empty (eraser).
- **The board:** a CSS grid of buttons, one per square, showing coordinates and the piece placed there. Click a square to place the selected piece, or drag a piece from the palette or from another square.
- **Resize:** − Row, + Row, − Column, + Column (3 to 20).
- **Live preview:** the full map (`warehouseLayoutSvg` with the draft).

When you **save**, the submit handler:

1. Copies only the edited zones into the real layout (other zones stay exactly as they were) and normalizes it.
2. **Relabels** products whose square changed between Rack, Shelf and Bin: the product stays and only the letter in its location code changes. The inner `relabel` does this and counts the changes.
3. **Warns** about locations the change would strand: previously fine, now outside a zone, turned into a walkway, emptied, or with no walkway reaching it. It asks for confirmation with a list of up to 10.
4. Applies the new layout and inventory, updates renamed stops in batches, re-plans every batch's route, records history ("Warehouse Grid Updated · Zone A · Start Zone A (8,4) · Routes Recalculated"), saves and redraws the Picking page.

`ensureDraftStartingPoint()` helper inside the function above

Makes sure the draft still has a valid starting point (in an active zone, on a Path square). If not, it places one at zone A's entry square.

`syncGlobalSettings()` helper inside the function above

Copies the three settings from the form into the draft, switches to zone A if the active zone was removed, and re-checks the starting point.

`placePiece(key, type, source)` helper inside the function above

Places a piece on a square. Doors are only allowed on the corridor edge, are made walkable and are saved in `draft.doors` (one per zone). The Starting Point moves S and makes the square walkable. The square holding S cannot be replaced by a container until S is moved. When a piece is dragged from another square, the old square is cleared. Empty removes the square's content. It marks the zone as edited and redraws the editor.

`beginPointerDrag(event, type, source)` helper inside the function above

Starts a drag for a piece. Only after the pointer moves 6 pixels (so a normal click is not mistaken for a drag), it shows a floating "ghost" of the piece and highlights the square under the pointer. On release it places the piece there and briefly ignores the click that follows the release.

`move(moveEvent)` helper inside the function above

While dragging: moves the ghost and updates the highlighted target square.

`end(upEvent)` helper inside the function above

At the end of a drag: removes the ghost and the highlight and places the piece on the square under the pointer.

`resizeZone(action)` helper inside the function above

Adds or removes the last row or column (keeping 3–20). Removing deletes the squares in that row or column. Then it re-checks the starting point and redraws.

`renderZoneEditor()` helper inside the function above

Redraws the editor: the zone tabs, the selected palette piece, the zone title and size, every board square (showing the starting point, the door, either chosen or "(auto)", and the placed pieces), the live preview, and the label of the Save button ("Save 2 Edited Zones"). Then it reconnects all the events.

`relabel(code)` helper inside the function above

Returns the location code with its container letter updated to the square's new type, and records the rename.

Called from: `renderPicking`

Activity history and demo reset

`renderHistory()` renderer.js · line 1351

Draws the Activity History page: a search box, an area filter (All Areas, Inventory, Orders, Picking, Layout), and one card per entry (`historyEntryMarkup`), or an empty message. The inner `applyFilters` hides the cards that do not match the search text or the chosen area.

`applyFilters()` helper inside the function above

Shows only the history cards that match the search and the area.

Called from: `render`

`historyEntryMarkup(entry)`

renderer.js · line 1359

One history card: the area icon, the action title, the area pill, the detail text and the date and time (in a `<time>` element). It carries `data-area` and `data-search` for filtering.

Called from: `renderHistory`

`resetDemoData()`

renderer.js · line 1365

The **Reset demo data** button. After confirmation, it replaces the whole state with fresh demo data (`defaultState`, upgraded), forgets the selected batch, resets the chart period, records the reset in history, saves and shows the Dashboard.

Called from: `startup code`

Starting the app (the last lines)

renderer.js · lines 1371–1376

```
1371  document.getElementById("reset-demo").addEventListener("click", resetDemoData);
1372
1373  hydrateStaticIcons();
1374  window.addEventListener("resize", () => { if (currentView === "dashboard")
      requestAnimationFrame(syncDashboardPanels); });
1375  updateReadyBadge();
1376  render();
```

When `renderer.js` has loaded, these lines run once: they connect the reset button, put icons into the sidebar (`hydrateStaticIcons`), keep the dashboard columns aligned when the window is resized, set the Orders badge and draw the first page (`render()`, the Dashboard). By this time `state` has already been loaded, at line 151 (`let state = loadState();`).

Part 5 · The Java route engine

What the engine does

The route engine solves one problem: *given the warehouse grid, the starting point and the stops of a batch, find the shortest walk that starts at S, visits every stop (fragile ones last) and returns to S.* This is a version of the famous **Travelling Salesman Problem**. It has two extra twists:

- **Several standing squares per stop.** A rack between two aisles can be picked from either aisle. The engine chooses both the *order* of the stops and *where to stand* at each one.
- **Fragile last.** Every fragile stop must come after every non-fragile stop.

The engine lives in package `com.lumina.engine`, seven files and 851 lines with no external libraries. The table shows what each class does, in the order a request flows through them.

Class	Lines	Job
`EngineServer`	53	Entry point. Reads one request line, answers one line, forever.
`Json`	209	Small JSON reader and writer (requests in, answers out).
`Warehouse`	93	The request as Java records: zones, start, corridor, stops.
`WarehouseGraph`	95	A graph of everywhere a picker can walk: squares and corridor points.
`ShortestPaths`	54	Dijkstra's algorithm: shortest distances and paths from one point.
`RouteOptimizer`	296	Chooses the best order and standing squares: Held-Karp (exact) or nearest neighbour + 2-opt (fast).
`RoutePlan`	51	The answer: order, legs, distance, comparison, method, time.

A real request and answer

These were captured while making the screenshots in this guide, for the first batch of the demo data (`B-S-261018-04`: 4 stops in zones A, B and C). The lists of Path squares and route points are shortened with "…".

```
REQUEST (one line on the engine's standard input)
{"zones": [{"name": "A", "rows": 8, "cols": 7,
            "paths": [[1,2],[1,4],[1,6],[2,2], … 28 walkway squares]},
           … zones B, C and D have the same grid …],
 "start": {"zone": "A", "row": 8, "col": 4},
 "corridor": [{"zone": "A", "row": 8, "col": 4, "pos": 9.338},
              {"zone": "B", "row": 8, "col": 4, "pos": 18.176},
              {"zone": "C", "row": 8, "col": 4, "pos": 27.014},
              {"zone": "D", "row": 8, "col": 4, "pos": 35.851}],
 "stops": [{"id": "PCK-STD-001@A-01-R01", "zone": "A", "row": 1, "col": 1, "last": false},
           {"id": "LAB-STD-001@B-03-R03", "zone": "B", "row": 3, "col": 3, "last": false},
           {"id": "PCK-STD-002@B-03-R01", "zone": "B", "row": 1, "col": 3, "last": false},
           {"id": "PCK-STD-003@C-03-R02", "zone": "C", "row": 2, "col": 3, "last": false}],
 "baselineOrder": ["PCK-STD-001@A-01-R01", "LAB-STD-001@B-03-R03",
                   "PCK-STD-002@B-03-R01", "PCK-STD-003@C-03-R02"],
 "returnToStart": true, "type": "optimize", "requestId": 1}

ANSWER (one line on the engine's standard output)
{"ok": true,
 "order": ["PCK-STD-003@C-03-R02", "PCK-STD-002@B-03-R01",
           "LAB-STD-001@B-03-R03", "PCK-STD-001@A-01-R01"],
 "unreachable": [],
```

```
"legs": [{"stop": "PCK-STD-003@C-03-R02",
          "points": [{"zone":"A","row":8,"col":4}, {"corridor":0}, {"corridor":1}, {"corridor":2},
                     {"zone":"C","row":8,"col":4}, {"zone":"C","row":7,"col":4}, …, {"zone":"C","row":2,"col":4}]},
         {"stop": "PCK-STD-002@B-03-R01", "points": [ … 17 points … ]},
         {"stop": "LAB-STD-001@B-03-R03", "points": [ … 3 points … ]},
         {"stop": "PCK-STD-001@A-01-R01", "points": [ … 18 points … ]},
         {"stop": null, "return": true, "points": [ … 10 points, ending at A row 8 col 4 … ]}],
"distance": 85.4, "baselineDistance": 85.4, "method": "exact", "micros": 31919,
"engine": "Lumina Route Engine 1.0 · Java 25", "requestId": 1}
```


How to read it:

- The first leg walks from S (zone A, square 8-4) onto the corridor, along it past zone B's door to zone C's door, into zone C and up column 4 to row 2. There the picker stands next to the rack at C-03-R02 (row 2, column 3). Its length is 1 + 8.838 + 8.838 + 1 + 6 = 25.676 steps. The corridor parts are the differences between the doors' `pos` values.
- `method: "exact"` means Held-Karp was used (4 stops ≤ 12), so 85.4 steps is the **shortest possible** round trip.
- `baselineDistance` is the built-in route's length for the same stops. Here it is also 85.4: the built-in route visits the same stops in the opposite direction, and a round trip has the same length both ways. On other batches Java's route is shorter, and the Picking page then shows "N fewer than built-in".
- `micros: 31919` means planning took about 32 milliseconds, including building the graph and every Dijkstra search. This was the engine's very first request, when Java is still warming up; the tests below show later plans taking a few milliseconds.

EngineServer.java: the program's entry point

`class EngineServer` — EngineServer.java · line 17

The class that holds `main`. It is `final` (no subclasses) and its version constant `VERSION = "1.0"` appears in every answer.

Called from: not called by another function here (used directly by the page, the system or a test).

`main(String[] args)` — EngineServer.java · line 20

The engine's main loop. It wraps standard input in a `BufferedReader` and standard output in a `PrintStream` (both UTF-8, output flushed after every line). Then it reads lines until the input closes, which is how Lumina stops it. Blank lines are skipped; every other line goes to `handle` and the answer is written as one JSON line with `Json.write`. Because the program keeps running, Java starts only once and later requests are fast.

Called from: not called by another function here (used directly by the page, the system or a test).

`handle(String line)` — EngineServer.java · line 32

Answers one request, and **always** returns an answer, even for errors.

1. Parses the line with `Json.parse` and reads `requestId` and `type` (default `"optimize"`).
2. A Java 21 `switch` expression picks the action: `"ping"` answers `{ ok: true }` (a health check); `"optimize"` runs `new RouteOptimizer(Warehouse.fromJson(request)).optimize().toJson()`; anything else answers "Unknown request type".
3. Any `RuntimeException` (bad JSON, a missing field) becomes `{ ok: false, error: "ExceptionName: message" }` instead of crashing the program.
4. Adds the engine name with the Java version (`"Lumina Route Engine 1.0 · Java 25"`) and echoes the `requestId`, so `route-engine.cjs` can match the answer to its request.

Called from: `main`, `protocol (EngineTest.java)`

Json.java: reading and writing JSON

The engine has no libraries, so it brings its own small JSON parser. It is a **recursive-descent parser**: one method per kind of value, which call each other for nested values. Objects become `Map<String, Object>` (a `LinkedHashMap`, which keeps the key order), arrays become `List<Object>`, numbers become `Double`, and `true`, `false` and `null` become Boolean and null.

`class Json`

Json.java · line 15

Holds the text being read (`text`) and the current reading position (`pos`). Its private constructor `Json(String text)` is used only by `parse`.

Called from: `parse`

`parse(String text)`

Json.java · line 22

Parses a whole JSON document: skips spaces, reads one value, skips spaces, and throws an error if anything is left over.

Called from: `handle (EngineServer.java)`, `viaJson (EngineTest.java)`, `protocol (EngineTest.java)`

`write(Object value)`

Json.java · line 32

Turns maps, lists, strings, numbers, booleans and null into JSON text, using a `StringBuilder` for speed.

Called from: `main (EngineServer.java)`, `viaJson (EngineTest.java)`, `protocol (EngineTest.java)`

`readValue()`

Json.java · line 40

Looks at the next character and decides what comes: `{` an object, `[` an array, `"` a string, `t` / `f` / `n` the words true/false/null, anything else a number.

Called from: `parse`, `readObject`, `readArray`

`readObject()`

Json.java · line 54

Reads `{ "key": value, … }`: after `{`, it repeatedly reads a key (which must be a string), a colon and a value (with `readValue`, so values can be nested objects), until `}`. A missing comma or brace gives a clear error.

Called from: `readValue`

`readArray()`

Json.java · line 74

Reads `[ value, value, … ]` in the same way.

Called from: `readValue`

`readString()`

Json.java · line 89

Reads a string up to the closing quote, translating escapes: `\"`, `\\`, `\/`, `\b`, `\f`, `\n`, `\r`, `\t` and `\uXXXX` (a character given by its four-digit hexadecimal code).

Called from: `readValue`, `readObject`

`readNumber()`

Json.java · line 114

Reads the characters that can appear in a number (`+-0123456789.eE`) and converts them with `Double.parseDouble`.

Called from: `readValue`

`readLiteral(String word, Object value)` Json.java · line 125

Checks that the text continues with `true`, `false` or `null` and returns the matching value.

Called from: `readValue`

`skipWhitespace()` Json.java · line 131

Moves past spaces, tabs and line breaks.

Called from: `parse`, `readObject`, `readArray`

`peek()` Json.java · line 135

The current character, without moving on (an error at the end of the text).

Called from: `readObject`, `readArray`, `next`

`next()` Json.java · line 140

The current character, then moves one position on.

Called from: `readObject`, `readArray`, `readString`, `expect`

`expect(char c)` Json.java · line 146

Reads one character and throws an error if it is not `c` (used for the colon after a key).

Called from: `readObject`

`error(String message)` Json.java · line 150

Makes an `IllegalArgumentException` that includes the position, for example "Expected ',' or '}' at position 57".

Called from: `readValue`, `readObject`, `readArray`, `readString`, `readNumber`, `readLiteral`, `peek`, `expect`

`writeValue(Object value, StringBuilder out)` Json.java · line 156

Writes one value, choosing by type with Java's **pattern matching for `instanceof`** (`value instanceof String s`). Numbers that are whole are written without ".0" (so 3.0 becomes 3); NaN and infinity become null, because JSON has no such values. Maps and lists are written recursively; any other object is written as its text.

Called from: `write`

`writeString(String s, StringBuilder out)` Json.java · line 191

Writes a string in quotes, escaping quotes, backslashes, line breaks, tabs and other control characters (as `\u00XX`).

Called from: `writeValue`

Warehouse.java: the request as Java records

A Java **record** is a compact class for plain data: you list the fields and Java writes the constructor, the getters, `equals`, `hashCode` and `toString`. Records are ideal here, because the warehouse description never changes once it is read.

`record Warehouse(List<Zone> zones, Square start, List<CorridorLink> corridor, List<Stop> stops, List<String> baselineOrder, boolean returnToStart)` Warehouse.java · line 20

The whole request: the zones, the starting point, the corridor doors, the stops, the built-in order (for comparison) and whether the route returns to the start.

Called from: `fromJson`

`record Square(String zone, int row, int col)` Warehouse.java · line 24

One grid square. Because it is a record, two squares with the same zone, row and column are *equal*, so a `Square` works as a key in a `HashMap` (used by `WarehouseGraph`).

Called from: `neighbour`, `square`, `WarehouseGraph (WarehouseGraph.java)`

`neighbour(int dRow, int dCol)` Warehouse.java · line 25

The square next to this one, moved by `dRow` rows and `dCol` columns (for example `neighbour(0, 1)` is the square to the right).

Called from: `WarehouseGraph (WarehouseGraph.java)`, `accessPoints (WarehouseGraph.java)`

`record Zone(String name, int rows, int cols, Set<String> walkways)` Warehouse.java · line 29

One zone's grid size and its set of walkway squares, stored as `"row-col"` texts.

Called from: `fromJson`

`isWalkway(int row, int col)` Warehouse.java · line 30

True when the square is inside the zone and is a walkway.

Called from: `WarehouseGraph (WarehouseGraph.java)`

`record CorridorLink(Square square, double position)` Warehouse.java · line 36

A door square that opens onto the corridor, with its position along the corridor in square-widths.

Called from: `fromJson`

`record Stop(String id, Square location, boolean last)` Warehouse.java · line 42

A location to visit: its id (`sku@location`), its storage square and whether it must come last (fragile).

Called from: `fromJson`

`fromJson(Map<String, Object> json)` Warehouse.java · line 46

Builds a `Warehouse` from the parsed request: every zone with its walkway set, the start square, the corridor links, the stops, the baseline order, and `returnToStart` (true unless the request explicitly says false). The private helpers below read values safely, so a missing value gives an empty list, an empty string or 0 instead of an error.

Called from: `handle (EngineServer.java)`, `viaJson (EngineTest.java)`

`square(Map<String, Object> m)` Warehouse.java · line 74

Reads `{ zone, row, col }` into a `Square` (null if missing).

Called from: `fromJson`

`list(Map<String, Object> m, String key)` Warehouse.java · line 80

The list stored under `key`, or an empty list.

Called from: `fromJson`

`str(Map<String, Object> m, String key)` Warehouse.java · line 85

The text under `key`, or "".

Called from: `fromJson`, `square`

`num(Object v)` Warehouse.java · line 90

The whole number in `v` (JSON numbers arrive as `Double`), or 0. It uses pattern matching: `v instanceof Number n ? n.intValue() : 0`.

Called from: `fromJson`, `square`

WarehouseGraph.java: where a picker can walk

A **graph** is a set of *nodes* (points) joined by *edges* (connections), each edge with a *cost*. Lumina's graph has a node for every walkway square, joined to its neighbouring walkway squares with cost 1. The zones are linked through corridor nodes: each door gets a corridor node (cost 1 to step onto it), and neighbouring doors along the corridor are joined with a cost equal to their distance apart. The graph is **undirected**: every edge can be walked both ways at the same cost.

`class WarehouseGraph` WarehouseGraph.java · line 16

Stores the nodes in a list, the edges as an **adjacency list** (for each node, the list of its edges), a map from square to node number, and the zones by name.

Called from: `RouteOptimizer (RouteOptimizer.java)`

`record Node(Warehouse.Square square, int corridorIndex)` WarehouseGraph.java · line 19

A node: a walkway square (`corridorIndex` −1) or a corridor point (`corridorIndex` = the door's number).

Called from: `WarehouseGraph`

`isCorridor()` WarehouseGraph.java · line 20

True for corridor nodes.

Called from: `point (RouteOptimizer.java)`, `addNode`

`record Edge(int to, double cost)` WarehouseGraph.java · line 24

One direction of an edge: the node it leads to and its cost.

Called from: `connect`

`WarehouseGraph(Warehouse warehouse)` WarehouseGraph.java · line 31

Builds the graph in three steps:

1. Adds a node for every walkway square of every zone.
2. Connects each walkway square to the walkway squares **to its right and below** (cost 1). Because every edge is added in both directions, this covers all four directions without counting any pair twice.

3. Adds a corridor node for each door that is a real walkway square, connects it to its square (cost 1), sorts the doors by their position along the corridor, and connects each door's corridor node to the next with cost equal to the difference in position.

Called from: `RouteOptimizer (RouteOptimizer.java)`

`addNode(Node node)` WarehouseGraph.java · line 63

Appends a node and an empty edge list, records walkway squares in the square-to-node map, and returns the new node's number.

Called from: `WarehouseGraph`

`connect(int a, int b, double cost)` WarehouseGraph.java · line 70

Adds an undirected edge: `a → b` and `b → a` with the same cost.

Called from: `WarehouseGraph`

`size()` WarehouseGraph.java · line 75

The number of nodes.

Called from: `ShortestPaths (ShortestPaths.java)`

`node(int index)` WarehouseGraph.java · line 76

The node with this number.

Called from: `pathPoints (RouteOptimizer.java)`

`edges(int index)` WarehouseGraph.java · line 77

The edges leaving this node.

Called from: `ShortestPaths (ShortestPaths.java)`

`indexOf(Warehouse.Square square)` WarehouseGraph.java · line 80

The node number of a walkway square, or −1 if the square is not a walkway.

Called from: `optimize (RouteOptimizer.java)`, `accessPoints`

`accessPoints(Warehouse.Square location)` WarehouseGraph.java · line 86

The walkway squares directly above, below, left and right of a storage square: the places where a picker can stand to pick from it. In the default layout a rack has one to three (racks in row 7 can also be reached from the walkway row below).

Called from: `optimize (RouteOptimizer.java)`

ShortestPaths.java: Dijkstra's algorithm

BFS is enough on the plain grid, but the corridor edges have different costs (for example 8.838), so the engine uses **Dijkstra's algorithm**, which finds shortest paths when edges have different (non-negative) costs.

`class ShortestPaths` ShortestPaths.java · line 13

Holds two arrays filled by the constructor: `distance[v]` (the shortest distance from the source to node `v`) and `previous[v]` (the node before `v` on that shortest path).

Called from: `searchFrom (RouteOptimizer.java)`

`ShortestPaths(WarehouseGraph graph, int source)` ShortestPaths.java · line 17

Runs Dijkstra from `source`:

1. Every distance starts at infinity, except the source at 0.
2. A **priority queue** always gives back the node with the smallest known distance.
3. For the node taken out, each edge is tried: if going through this node makes a neighbour closer (`distance[node] + cost < distance[neighbour]`), the neighbour's distance and `previous` are updated and it is added to the queue. This step is called *relaxing* the edge.
4. A node can be in the queue several times. An entry whose distance is larger than the node's best known distance is out of date and is skipped (*lazy deletion*), which is simpler than updating entries inside the queue.

Running time is about O(E log V) for V nodes and E edges, a few milliseconds even for large warehouses.

Called from: `searchFrom (RouteOptimizer.java)`

`distanceTo(int node)` ShortestPaths.java · line 42

The shortest distance from the source to `node` (infinity if unreachable).

Called from: `buildDistanceTables (RouteOptimizer.java)`

`reaches(int node)` ShortestPaths.java · line 44

True when `node` can be reached from the source.

Called from: `optimize (RouteOptimizer.java)`, `pathTo`

`pathTo(int target)` ShortestPaths.java · line 47

The actual path: it follows `previous` back from the target to the source, then reverses the list. Empty if unreachable.

Called from: `pathPoints (RouteOptimizer.java)`

RouteOptimizer.java: choosing the best route

This is the "brain" of the engine.

`class RouteOptimizer` RouteOptimizer.java · line 26

Holds the warehouse, its graph, a cache of Dijkstra searches, and the tables filled by `optimize`: the reachable stops, their access squares (`options`), the distance tables, the fragile flags and whether to count the walk back to S. `EXACT_LIMIT = 12` is the largest batch solved exactly.

Called from: `handle (EngineServer.java)`, `handcraftedLuminaLayout (EngineTest.java)`, `randomizedExactness (EngineTest.java)`, `heuristicQuality (EngineTest.java)`, `performance (EngineTest.java)`

`record Tour(int[] order, int[] access, double cost)` RouteOptimizer.java · line 30

A candidate route: the order of the stops, which access square is used at each, and the total cost.

Called from: `optimize`, `heldKarp`, `bestAccessFor`

`RouteOptimizer(Warehouse warehouse)`

RouteOptimizer.java · line 46

Builds the graph for the warehouse. A second, package-private constructor `RouteOptimizer(warehouse, exactLimit)` lets the tests choose the limit, for example 0 to force the fast method on small batches so it can be compared with the exact one.

Called from: `handle (EngineServer.java)`, `handcraftedLuminaLayout (EngineTest.java)`, `randomizedExactness (EngineTest.java)`, `heuristicQuality (EngineTest.java)`, `performance (EngineTest.java)`

`searchFrom(int node)`

RouteOptimizer.java · line 55

Dijkstra from `node`, computed once and remembered (`computeIfAbsent` on a `HashMap`). The same access square is needed many times, so caching saves a lot of work.

Called from: `optimize`, `buildDistanceTables`, `pathPoints`

`optimize()`

RouteOptimizer.java · line 59

Plans the route:

1. Finds the start node. If S is not a walkway square, it returns a failed plan ("The starting point is not on a walkway square") with every stop unreachable.
2. Runs Dijkstra from S. For each stop it keeps only the access squares S can reach; a stop with none is **unreachable** and is set aside.
3. Builds the distance tables (`buildDistanceTables`).
4. Chooses the method: up to 12 stops **Held-Karp** (exact), more stops **nearest neighbour + 2-opt** (fast).
5. Computes the cost of the built-in order (`bestAccessFor(baselineOrder())`) for comparison.
6. Returns a `RoutePlan` with the order (reachable stops first, then unreachable ones), the legs, the distances, the method and the time taken in microseconds.

Called from: `handle (EngineServer.java)`, `handcraftedLuminaLayout (EngineTest.java)`, `randomizedExactness (EngineTest.java)`, `heuristicQuality (EngineTest.java)`, `performance (EngineTest.java)`

`buildDistanceTables(ShortestPaths start)`

RouteOptimizer.java · line 96

Precomputes every distance the optimizer will need. `fromStart[i][a]` is the distance from S to access square `a` of stop `i`; it is also the walk back, because the graph is undirected. `between[i][a][j][b]` is the distance from access square `a` of stop `i` to access square `b` of stop `j`. One Dijkstra run per access square fills a whole row of the table.

Called from: `optimize`

`allowed(int visited, int next, int lastMask)`

RouteOptimizer.java · line 115

The **fragile-last rule**, using bit masks: each stop is one bit, `visited` holds the stops already visited and `lastMask` the fragile stops. A fragile stop may be visited only when every non-fragile stop is done (all non-fragile bits are set in `visited`). A non-fragile stop may be visited only while no fragile stop has been visited yet.

Called from: `heldKarp`

`heldKarp()`

RouteOptimizer.java · line 121

The **Held-Karp algorithm** (dynamic programming), which finds the truly shortest route. Trying every order is impossible for 12 stops: that is 12! ≈ 479 million orders, before even choosing standing squares. Held-Karp instead remembers the best partial routes:

- `best[mask][stop, access]` = the length of the shortest walk that starts at S, visits exactly the stops in `mask` (a bit set) and ends standing at that access square of that stop.
- **Start:** for each stop the fragile rule allows first, `best[{i}][i, a] = fromStart[i][a]`.
- **Grow:** for every mask, every end state and every unvisited stop `j` that `allowed` permits, try extending: `best[mask ∪ {j}][j, b] = min(…, best[mask][i, a] + between[i][a][j][b])`. A `parent` table remembers where each best value came from.
- **Close:** for the full mask, add the walk back to S (`returnWeight × fromStart`) and take the smallest total.
- **Rebuild:** follow `parent` backwards from the best end state to recover the order and the access squares.

With n stops and up to W access squares per stop there are $2^n \times n \times W$ states, and each tries $n \times W$ extensions: about $2^n \times (n \cdot W)^2$ steps. For 12 stops and W = 2 that is 4,096 × 576 ≈ 2.4 million steps, which takes milliseconds. Each extra stop doubles the work, which is why bigger batches use the fast method.

Called from: `optimize`

`nearestNeighbour()`

RouteOptimizer.java · line 176

The fast method's first step: a **greedy** route. Starting at S, it always walks to the nearest unvisited stop (over all its access squares) that the fragile rule allows. Fragile stops are skipped while non-fragile ones remain. The result's standing squares are then improved with `bestAccessFor`.

Called from: `optimize`

`improveWithTwoOpt(Tour tour)`

RouteOptimizer.java · line 202

2-opt improvement: it takes a stretch of the route between positions i and k and reverses it. If the route gets shorter, it keeps the change and searches again, for at most 50 rounds or until nothing improves. Reversing a stretch removes "crossings" in the walk. A stretch is only reversed when both ends are in the same group, so fragile stops never move before non-fragile ones. The tests measured this method at 0.2% longer than the best route on average (worst case 11.6%) for batches of 8–12 stops.

Called from: `optimize`

`bestAccessFor(int[] order)`

RouteOptimizer.java · line 222

For a **fixed** order of stops, chooses the best standing square at each stop, again by dynamic programming. `cost[p][b]` is the shortest walk up to position `p` standing at access square `b`; it is computed from the previous position's costs. At the end it adds the walk back to S, picks the best final square and follows the `back` table to recover the others. It is used by the fast method, by the baseline comparison and by 2-opt for every candidate.

Called from: `optimize`, `nearestNeighbour`, `improveWithTwoOpt`

`baselineOrder()`

RouteOptimizer.java · line 255

The built-in route's order (sent by the page as `baselineOrder`), limited to reachable stops. Stops missing from it are added at the end, and a stable sort puts fragile stops last, since the built-in routing follows the same rule. Used to report how much the engine saves.

Called from: `optimize`

`buildLegs(Tour tour)`

RouteOptimizer.java · line 266

Turns the best route into walking legs: from S to the first stop's standing square, from there to the next, and so on, plus the return leg to S. Each leg's points come from `pathPoints`.

Called from: `optimize`

`pathPoints(int from, int to)` RouteOptimizer.java · line 279

The shortest path between two nodes (from the cached Dijkstra search) as a list of points.

Called from: `buildLegs`

`point(WarehouseGraph.Node node)` RouteOptimizer.java · line 285

Turns a node into a JSON point: `{ "corridor": index }` for a corridor node, or `{ "zone", "row", "col" }` for a square.

Called from: `pathPoints`

RoutePlan.java: the answer

`record RoutePlan(boolean ok, String error, List<String> order, List<String> unreachable, List<Leg> legs, double distance, double baselineDistance, String method, long micros)` RoutePlan.java · line 20

The engine's result: whether it worked, an error if not, the visiting order (unreachable stops last), the unreachable stops, the legs, the route length and the built-in route's length (both in square-widths, including the walk back), the method (`"exact"`, `"heuristic"` or `"none"`) and the planning time in microseconds.

Called from: `optimize (RouteOptimizer.java)`, `failed`

`record Leg(String stopId, List<Map<String, Object>> points, boolean isReturn)` RoutePlan.java · line 24

One walk: to a stop (its id), or back to the start (`isReturn`), as a list of points.

Called from: `buildLegs (RouteOptimizer.java)`

`failed(String error, List<String> unreachable)` RoutePlan.java · line 26

A ready-made failed plan, with every stop unreachable and no legs.

Called from: `optimize (RouteOptimizer.java)`

`toJson()` RoutePlan.java · line 30

Converts the plan to a `Map` for `Json.write`, keeping the keys in a fixed order. Distances are rounded to one decimal (85.4). A return leg gets `"return": true`.

Called from: `handle (EngineServer.java)`

EngineTest.java: how we know it is correct

The tests are a separate program (not shipped in the app). They check the engine against an **independent** solution built in a completely different way, so a bug in the engine cannot hide by also being in the checker. The run for this guide printed:

```
exactness: 500 random warehouses matched brute force
heuristic: average 0.2% longer than optimal, worst 11.6% (120 batches of 8–12 stops)
performance: 12 stops exact in 4 ms; 40 stops heuristic in 8 ms
22902 checks, 0 failures
ALL TESTS PASSED
```

`class EngineTest`

EngineTest.java · line 14

Counts checks and failures. Its random number generator has a fixed seed (20261016), so every run tests the same "random" warehouses and results can be repeated.

Called from: not called by another function here (used directly by the page, the system or a test).

`check(boolean ok, String message)`

EngineTest.java · line 18

Counts one check; a failed check is counted and printed (the first 20).

Called from: `checkLegs`, `handcraftedLuminaLayout`, `randomizedExactness`, `heuristicQuality`, `performance`, `protocol`

`main(String[] args)`

EngineTest.java · line 23

Runs the five test groups, prints the totals, and exits with code 0 (all passed) or 1 (failures), so a build script can detect problems.

Called from: not called by another function here (used directly by the page, the system or a test).

`randomRequest(int zoneCount, int rows, int cols, int stopCount, double walkwayShare)`

EngineTest.java · line 36

Makes a random warehouse as JSON, like Lumina sends: random walkway squares (the bottom row is always a walkway), corridor doors, random stops on storage squares (30% fragile), a reversed baseline order and, 80% of the time, a return to the start.

Called from: `randomizedExactness`, `heuristicQuality`, `performance`, `protocol`

`viaJson(Map<String, Object> request)`

EngineTest.java · line 75

Writes the request to JSON text and parses it back before building the `Warehouse`, so the tests also exercise the JSON code exactly as in real use.

Called from: `handcraftedLuminaLayout`, `randomizedExactness`, `heuristicQuality`, `performance`

`class Reference`

EngineTest.java · line 82

An independent model of the warehouse. It builds its own graph and computes the distance between **every pair** of points with the **Floyd-Warshall** algorithm, a completely different method from the engine's Dijkstra.

Called from: `handcraftedLuminaLayout`, `randomizedExactness`

`Reference(Map<String, Object> req)`

EngineTest.java · line 88

Builds the reference graph from the request: walkway squares, corridor points and their edges, then runs Floyd-Warshall (three nested loops over all nodes).

Called from: `handcraftedLuminaLayout`, `randomizedExactness`

`link(int a, int b, double w)`

EngineTest.java · line 128

Adds an edge in both directions, keeping the cheaper one if it already exists.

Called from: `Reference`

`access(String zone, int r, int c)`

EngineTest.java · line 130

The reference version of `accessPoints`: walkway nodes next to a storage square.

Called from: not called by another function here (used directly by the page, the system or a test).

`num(Object o)`

EngineTest.java · line 140

Reads a JSON number as an `int`.

Called from: `Reference`, `bruteForce`, `checkLegs`, `samePlace`, `stepCost`, `protocol`

`bruteForce(Map<String, Object> req, Reference ref)`

EngineTest.java · line 144

The truly shortest route found by trying **every** order and every standing square, obeying the fragile rule and the walk back. It uses `permute`.

Called from: `handcraftedLuminaLayout`, `randomizedExactness`

`permute(...)`

EngineTest.java · line 161

Recursive brute-force search with **branch and bound**: a partial route already longer than the best complete route found so far is abandoned (`if (cost >= best[0]) return`), which keeps brute force fast enough for up to 6 stops.

Called from: `bruteForce`

`checkLegs(Map<String, Object> req, RoutePlan plan, String label)`

EngineTest.java · line 178

Checks that the answer is a **real walk**. Every leg starts where the last one ended; each step goes to an adjacent walkway square or along the corridor; fragile stops come after all others; each leg ends next to its stop; the steps add up exactly to the reported distance; and there is a return leg exactly when one was requested.

Called from: `handcraftedLuminaLayout`, `randomizedExactness`, `heuristicQuality`, `performance`

`samePlace(Map<String, Object> a, Map<String, Object> b)`

EngineTest.java · line 216

True when two points are the same square.

Called from: `checkLegs`, `stepCost`

`stepCost(Map<String, Object> a, Map<String, Object> b, List<Object> corridor)`

EngineTest.java · line 221

The cost of one step between two points (1 between neighbouring squares or between a door square and the corridor, the position difference along the corridor), or −1 if the step is impossible.

Called from: `checkLegs`

`handcraftedLuminaLayout()`

EngineTest.java · line 235

Tests Lumina's own default zone (8 × 7, walkways on the bottom row and the even columns). The route must be optimal and must return to the start. Marking the nearer stop fragile must make it come last. A stop with no walkway next to it must be reported as unreachable.

Called from: `main`

`randomizedExactness(int instances)`

EngineTest.java · line 262

For 500 random small warehouses, checks that the engine's distance equals brute force exactly, that it is never longer than the built-in route, that every stop is in the order, and that the legs are a real walk.

Called from: `main`

`heuristicQuality(int instances)` EngineTest.java · line 280

For 120 batches of 8–12 stops, runs both methods: the fast method must never beat the exact one (which would mean a bug in the exact one), its walks must be valid, and its average gap must be under 10%.

Called from: `main`

`performance()` EngineTest.java · line 296

Times a 12-stop exact plan (must take under 3 seconds) and a 40-stop fast plan (under 5 seconds) on 4 zones of 20 × 20 squares. Measured: 4 ms and 8 ms.

Called from: `main`

`protocol()` EngineTest.java · line 312

Tests the line protocol through `EngineServer.handle`: the `requestId` is echoed; the answer is valid JSON with legs and the engine name; malformed JSON gives an error answer; `ping` works; and a bad starting point is reported.

Called from: `main`

Part 6 · Walkthrough: the life of one order

This part follows one order through Lumina and names every function that runs on the way. It is a good backbone for a class demo, because you can click through it live while explaining.

1. Starting the app

1. Electron runs `main.cjs`: it sets the data folder, creates the `RouteEngine` (Java is not started yet) and calls `createWindow()`.
2. The window loads `index.html`. `preload.cjs` adds `window.lumina.optimizeRoute`.
3. `renderer.js` runs from top to bottom. On line 151 `loadState()` reads localStorage and upgrades old data (`upgradeState`, the `migrate…` functions), or creates the demo data with `defaultState()`.
4. The last lines run `hydrateStaticIcons()`, `updateReadyBadge()` and `render()`, which calls `renderDashboard()`. The dashboard counts the orders (`orderDisplayStatus`), builds the notifications (`buildAlerts`) and draws the chart (`drawChart`).

2. Creating an order

1. On the Orders page (`navigate("orders")` → `renderOrders()`), **New order** calls `openOrderModal()`.
2. As items are chosen, `refreshHints` shows "N available" using `freeStockForNewOrders()`.
3. **Create order**: the form is checked, and `addOrder(...)` gives it the next number (`nextOrderNumber`), status `ready` and `events.createdAt`.
4. `recordHistory("Order Created", …)`, then `saveState()` writes everything to localStorage, then `navigate("orders", { highlight })` redraws the table and flashes the new row (`applyHighlight`).

3. Batching

1. **Create automatic batches** calls `createAutomaticBatches()`.
2. Pending orders are sorted (`sortReadyOrders`) and checked (`orderBlockers` with `pickableStockBySku`). Our order passes, so its units are counted as taken.
3. It joins a group with the same handling class, priority and due day, and a cart (at most 4 orders and 60 load units, measured by `orderLoad`).
4. The cart goes to the least-busy picker (`batchLoad` of their open batches). Its stops come from `allocateStops` (primary location first, reservations respected) and are put in walking order by `optimizeRouteStops` → `buildEfficientRoutePlan` (BFS with `shortestPathToAny`).
5. The order becomes `batched` with a code like `B-S-261018-04`. History, save, and `navigate("picking")`.

4. Planning the best route with Java

1. `renderPicking()` sees no valid plan (`hasValidPlan` is false), draws the built-in route (`builtinRoutePlan`) and calls `requestRoutePlan(batch)`.
2. `buildRouteRequest` describes the grid, S, the corridor doors (`corridorLinks`) and the stops. `window.lumina.optimizeRoute(request)` sends it through IPC to `main.cjs`, which calls `routeEngine.request(...)`.
3. The first time, `RouteEngine.start()` launches Java. The request is written as one line with `requestId: 1`.

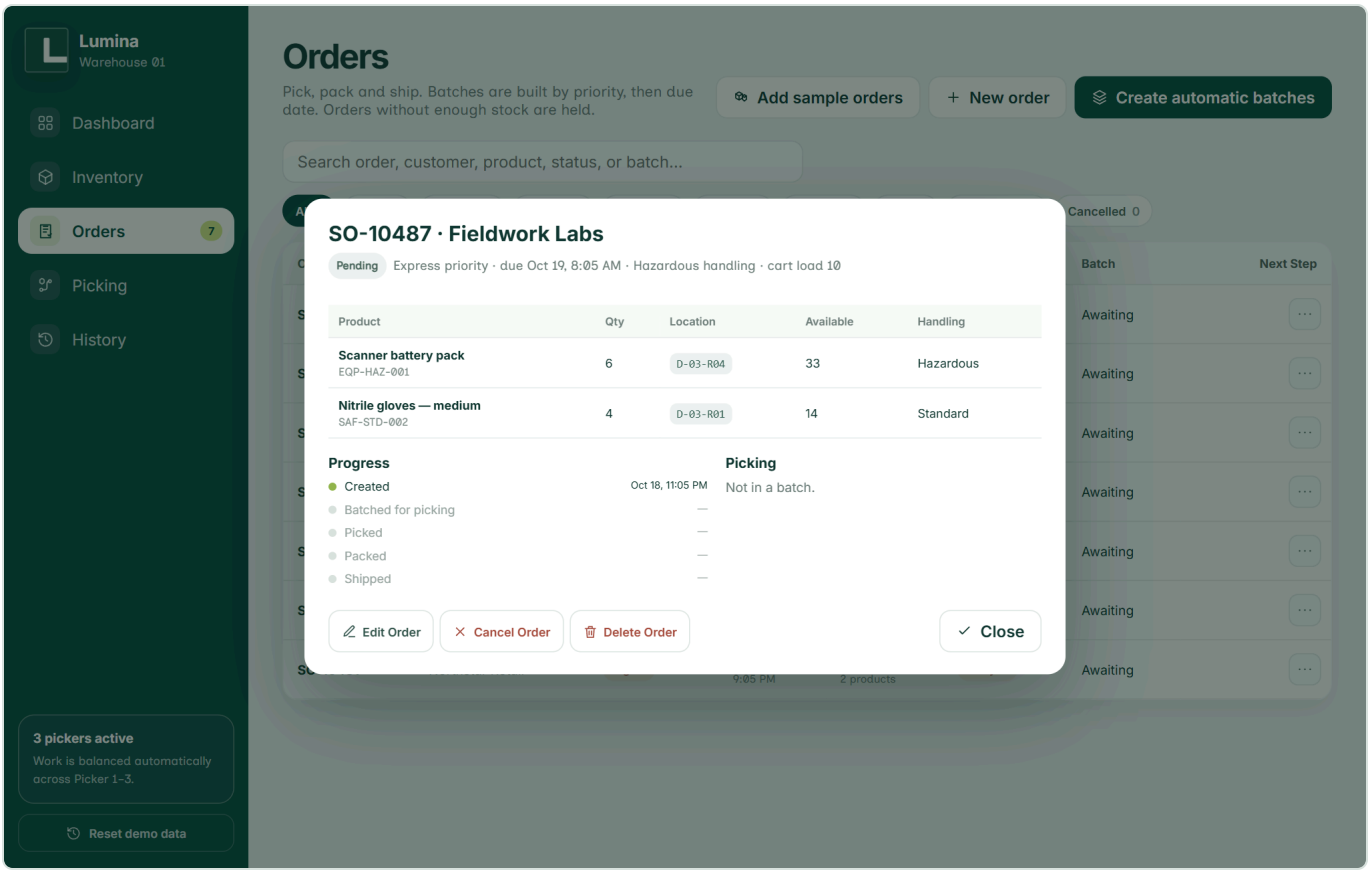
4. In Java: `EngineServer.main` reads the line, then `handle` calls `Json.parse`, then `Warehouse.fromJson`, then `new RouteOptimizer(...)` (which builds the `WarehouseGraph`). `optimize()` runs Dijkstra (`ShortestPaths`) from S and from every access square, fills the distance tables, runs `heldKarp()` (4 stops ≤ 12), builds the legs and returns a `RoutePlan`. `toJson()` and `Json.write` send back one line.
5. `RouteEngine.receive` matches `requestId: 1` and resolves the promise. In the page, `finish` reorders the stops, stores `batch.plan` and saves; `renderPicking()` redraws the map with Java's route and the chip "Java route engine · 85 steps round trip · shortest possible".

5. Picking

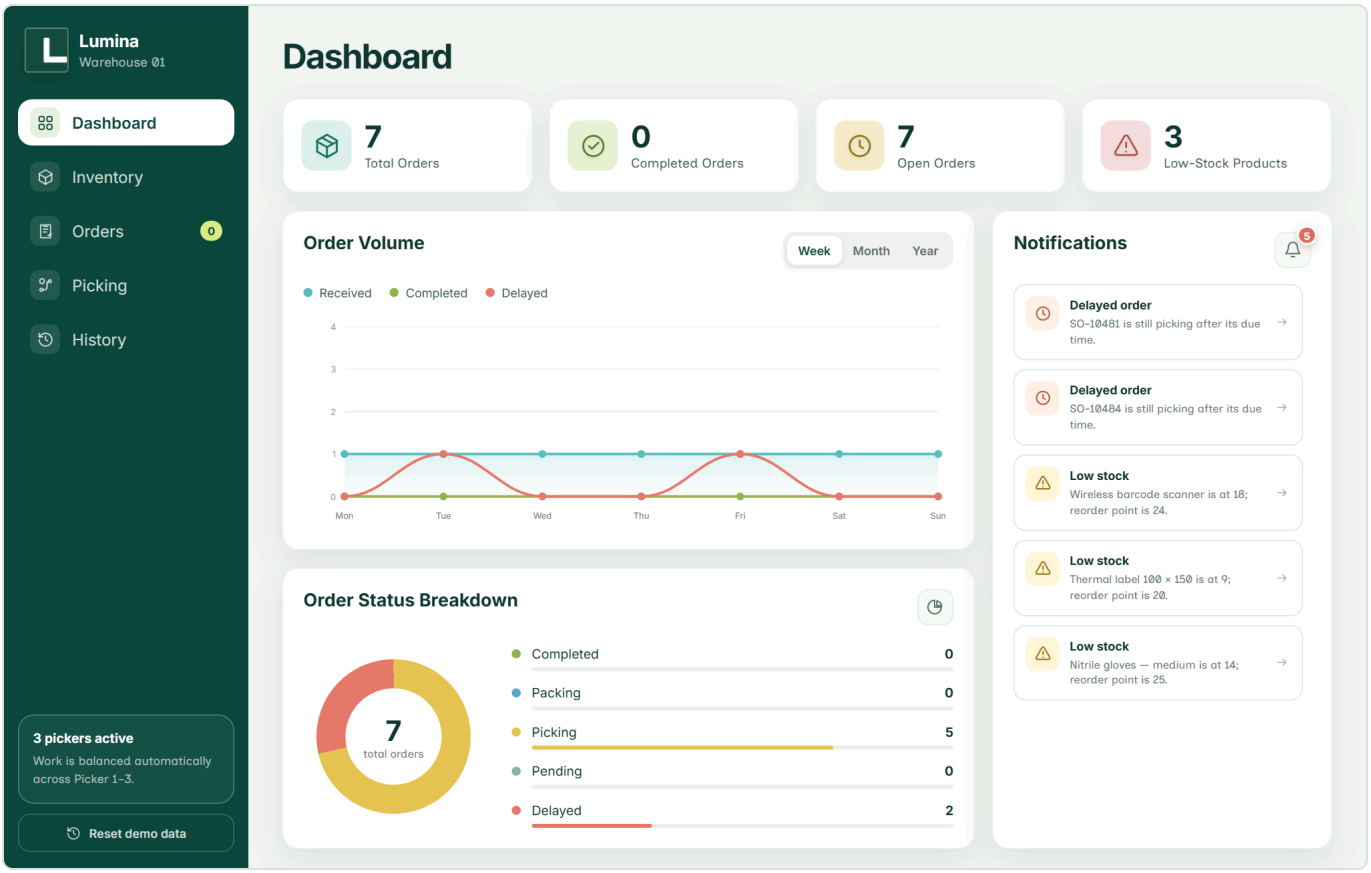
1. The picker clicks **Mark picked** on each stop: `toggleStopPicked` updates the progress (`batchPickCounts`) and the batch status, saves and redraws. The next stop moves along the route.
2. If a shelf has fewer units, **Short Pick** calls `openShortPickModal`, which records the units found.
3. When every stop is ticked, the banner says to walk back to S. **Complete Batch** is now enabled.

6. Completing, packing and shipping

1. **Complete Batch** calls `completeBatch(batch)`. Units come off the shelves ("Picked" movements, `recordMovement`); short-picked shelves are corrected ("Count correction"); orders are filled most urgent first. Our order becomes `picked`. Leftovers go back on the shelf. History and save follow.
2. On the Orders page our order shows **Pack** (`orderActionsMarkup`). `packOrder` sets it to `packed`, and `shipOrder` sets it to `shipped`, shown as **Completed**. Each step records its time in `order.events`, which the details window shows as a timeline (`openOrderDetails`).
3. The Dashboard now counts one more completed order. The History page lists every step (`renderHistory`).



The order details window: lines with locations and available stock, the timeline of events, and the picking information.



The Dashboard after batching: orders now appear under Picking in the status breakdown, and the notifications have changed.

Part 7 · Presenting Lumina in class

A suggested slide plan (10–15 minutes)

#	Slide	What to say	Show
1	Lumina in one sentence	A warehouse app that turns customer orders into the shortest picking routes for three pickers.	Cover screenshot
2	The problem	Pickers waste time walking. Orders must be grouped safely (hazardous apart, fragile on top) and stock must stay correct.	Warehouse map
3	Architecture	Three processes: the page (UI and rules), the Electron main process, and the Java route engine, connected by IPC and JSON lines.	Part 2 diagram
4	Data	One `state` object saved to localStorage; location codes like `A-03-R04`; a grid of Path / Rack / Shelf / Bin squares.	Data table
5	Business rules	Capacities, cart limit 60, 4 orders per cart, handling classes, fragile last, holds, reservations.	Rules table
6	Batching	`createAutomaticBatches`: sort, check stock, group, fill carts, balance pickers.	Picking page
7	Built-in routing	Breadth-first search on the grid, nearest stop next.	`shortestPathToAny` code
8	The Java engine	Graph + Dijkstra + Held-Karp (exact up to 12 stops) or nearest neighbour + 2-opt.	Request / answer
9	Correctness	22,902 automated checks, compared with brute force on 500 random warehouses.	Test output
10	Live demo	Create order → batch → route → pick → complete → pack → ship.	The app
11	Limits and ideas	See below.	—

A live demo script

1. Click **Reset demo data** so everyone sees the same starting point.
2. **Dashboard:** point out the Delayed and urgent notifications, then click one; it jumps to the order and highlights it.
3. **Inventory:** open "⋯" → Locations & Transfers on a product to show stock per location and the map.
4. **Orders: New order** for a customer, add two items, and show the "N available" hints. Create it.
5. **Create automatic batches:** hazardous orders get their own carts (red badges), and the batches are spread over Pickers 1–3.
6. **Picking:** show the route from S and back, the chip "Java route engine ... shortest possible", the next-stop banner and the fragile tag.
7. Tick the stops (use **Short** once to show a short pick), then **Complete Batch**.
8. **Orders: Pack** and **Ship** the order, then open its details to show the timeline.
9. **History:** every action was logged.
10. Optional: **Edit Warehouse Layout**, turn a path into a rack, and show that Lumina warns about unreachable locations.

Questions you may be asked

Why is the route engine written in Java and not in JavaScript?

Finding the exact shortest route is heavy computation (Held-Karp explores millions of states). Running it as a separate Java program keeps the window responsive, and Java is fast at this kind of number crunching. The page keeps a JavaScript fallback (`buildEfficientRoutePlan`), so the app works even without Java.

How do the app and Java talk to each other?

Through the Java process's standard input and output, one JSON object per line. `route-engine.cjs` writes a request with a `requestId` and Java echoes the id in its answer, so answers can be matched even when several requests are waiting. Requests time out after 15 seconds.

Is the Java route always the shortest?

For up to 12 stops, yes: Held-Karp is an exact algorithm, and the tests compare it with brute force on 500 random warehouses. Above 12 stops it uses nearest neighbour + 2-opt, which the tests measured at 0.2% longer than the best route on average (worst case 11.6%), in milliseconds.

What is the difference between BFS and Dijkstra here?

Both find shortest paths. BFS works when every step costs the same, which is true inside one zone, so the JavaScript side uses it. Dijkstra handles different costs; the corridor between zones has longer edges (like 8.8 squares), so the Java engine uses Dijkstra.

Why are fragile items picked last?

So they end up on top of the cart and nothing heavy is put on them. Both routing methods enforce it: the built-in one plans the fragile group after the others, and Java's `allowed()` function forbids a fragile stop before all normal stops are done.

How does Lumina decide which orders share a cart?

Same handling class (standard, hazardous, cold chain), same priority and same due day, then at most 4 orders and 60 load units per cart (oversized items count 5 each). Hazardous and cold-chain goods never share a cart with other goods.

How are pickers balanced?

Each new batch goes to the picker with the least unfinished work, measured in load units of their open batches, not just the number of batches.

Where is the data stored? Is there a database?

No database server. The whole state is one JSON text in the browser's localStorage, which Electron stores in `%APPDATA%\Wayline Warehouse`. It is simple and works offline, but it is for one computer only: two people cannot share the same data.

What happens when stock is short?

Orders without enough reachable stock are *held* and listed with the reason (`heldOrders`). If a picker finds fewer units than expected, the short pick corrects the shelf count when the batch is completed, orders that cannot be filled go back to Pending, and unused units return to the shelf.

How is the app protected?

The window runs with context isolation, a sandbox and no Node.js access; the page can only call `window.lumina.optimizeRoute`. A Content-Security-Policy blocks outside scripts, and every user-typed text goes through `escapeHtml` before being shown, which prevents HTML injection.

Is the Order Volume chart real data?

It is illustrative: `distribute` spreads the current totals evenly over the days, weeks or months. It does not use the actual order dates. That is a good improvement to mention.

Limits and ideas for improvement

- **One computer only:** localStorage is local. A real deployment would use a database and a server so several users share the data.
- **The chart** spreads totals instead of using real dates (see `distribute`).
- **One big file:** `renderer.js` holds about 160 functions. Splitting it into modules (inventory, orders, picking, routing, map) would make it easier to maintain and test.
- **Tests:** the Java engine has thorough tests; the JavaScript rules (batching, completing, CSV import) have none yet.
- **Native dialogs:** some messages use `alert` and `confirm`, which block the app; custom windows would be friendlier.
- **Routing:** the built-in route is greedy; it could use the same exact algorithm in JavaScript for small batches.

Glossary

Term	Meaning in Lumina
SKU	Stock keeping unit: the product code, such as `PCK-STD-001`.
Location code	Where a product is stored, such as `A-03-R04` (zone A, column 3, Rack, row 4).
Primary location	A product's first location; picking takes from it first.
On hand / Reserved / Available	Units on the shelf / units promised to open batches / on hand − reserved.
Batch	One picking cart: up to 4 orders, picked by one picker along one route.
Stop	One place to visit in a batch and the units to take there.
Short pick	Finding fewer units on the shelf than expected.
Held order	A pending order that cannot be batched (missing, unreachable or short stock).
Load unit	Cart space: 1 per item, 5 per oversized item; a cart holds 60.
Handling class	Standard, Hazardous or Cold chain; decides which orders may share a cart.
Starting point (S)	Where every route starts and ends: the cart drop-off.
Dispatch corridor / door	The lane joining the zones / each zone's opening onto it.
IPC	Inter-process communication: messages between Electron's page and main process.
BFS	Breadth-first search: shortest paths when every step costs the same.
Dijkstra	Shortest paths when steps have different (non-negative) costs.
Held-Karp	Dynamic programming that finds the exact shortest round trip (TSP) in about $2^n \cdot n^2$ steps.
2-opt	Improves a route by reversing stretches of it while that shortens it.
Record (Java)	A compact class for plain, unchanging data.
localStorage	The browser's built-in key-value storage, used to save Lumina's state.

Appendix · Index of every function

Every function, method and type in the code, alphabetically, with its file and line number.

access	EngineTest.java 130
accessPoints	WarehouseGraph.java 86
addNode	WarehouseGraph.java 63
addOrder	renderer.js 725
adjacentPathKeys	renderer.js 1034
alertMarkup	renderer.js 362
allocateStops	renderer.js 854
allowed	RouteOptimizer.java 115
app	renderer.js 2
applyHighlight	renderer.js 208
availableStockBySku	renderer.js 643
BASE_CATEGORIES	renderer.js 179
baselineOrder	RouteOptimizer.java 255
batchHandlingClass	renderer.js 872
batchLoad	renderer.js 106
batchPickCounts	renderer.js 905
bestAccessFor	RouteOptimizer.java 222
bindLocationMap	renderer.js 634
bindOrderActions	renderer.js 694
boundaryPathKeys	renderer.js 1035
bruteForce	EngineTest.java 144
buildAlerts	renderer.js 338
buildDistanceTables	RouteOptimizer.java 96
buildEfficientRoutePlan	renderer.js 1063
buildLegs	RouteOptimizer.java 266
buildRouteRequest	renderer.js 1131
builtinRoutePlan	renderer.js 1176
cancelBatch	renderer.js 989
cancelOrder	renderer.js 817
capacityOf	renderer.js 75
capitalize	renderer.js 851
CART_CAPACITY	renderer.js 69
chartPeriod	renderer.js 153
check	EngineTest.java 18
checkLegs	EngineTest.java 178
closeModal	renderer.js 555
closeRowMenus	renderer.js 554
completeBatch	renderer.js 999
composeLocation	renderer.js 632
connect	WarehouseGraph.java 70
constructor	route-engine.cjs 9
CONTAINER_CAPACITY	renderer.js 68
containerTypeOf	renderer.js 74
convertTwoSideZoneToGrid	renderer.js 53
corridorLaneSvg	renderer.js 1216
CorridorLink	Warehouse.java 36
corridorLinks	renderer.js 1116
createAutomaticBatches	renderer.js 873
createWindow	main.cjs 18
createZoneLayout	renderer.js 51
csvValue	renderer.js 564
currentView	renderer.js 152
decodeCsvBytes	renderer.js 567
defaultState	renderer.js 126
defaultWarehouseLayout	renderer.js 55
deleteOrder	renderer.js 826
deleteProduct	renderer.js 556
detachOrderFromBatch	renderer.js 971
distanceTo	ShortestPaths.java 42
distribute	renderer.js 302
downloadInventoryTemplate	renderer.js 565
drawChart	renderer.js 304
due	renderer.js 50
Edge	WarehouseGraph.java 24
edges	WarehouseGraph.java 77
engineDir	main.cjs 10
engineRouteSvg	renderer.js 1229
EngineServer	EngineServer.java 17
engineStatusMarkup	renderer.js 1164
EngineTest	EngineTest.java 14
error	Json.java 150
escapeHtml	renderer.js 173
expect	Json.java 146
fail	route-engine.cjs 59
failed	RoutePlan.java 26
filterChips	renderer.js 215
formatDate	renderer.js 174
formatTime	renderer.js 175
freeStockForNewOrders	renderer.js 724
fromJson	Warehouse.java 46
fs	route-engine.cjs 5
generateSampleOrders	renderer.js 797
gridKey	renderer.js 1031
gridPosition	renderer.js 1032
handcraftedLuminaLayout	EngineTest.java 235
handle	EngineServer.java 32
handlingBadge	renderer.js 906
hashText	renderer.js 1128
hasValidPlan	renderer.js 1130
heldKarp	RouteOptimizer.java 121
heldOrders	renderer.js 668
heuristicQuality	EngineTest.java 280

historyEntryMarkup renderer.js 1359
HOLD_STATUSES renderer.js 180
hydrateStaticIcons renderer.js 46
icon renderer.js 42
ICON_PATHS renderer.js 6
importInventoryCsv renderer.js 569
improveWithTwoOpt RouteOptimizer.java 202
indexOf WarehouseGraph.java 80
isContainerReachable renderer.js 1052
isCorridor WarehouseGraph.java 20
isCorridorEdge renderer.js 1040
isLocationWithinLayout renderer.js 633
isLowStock renderer.js 182
isOrderDelayed renderer.js 185
isOrderOpen renderer.js 110
isOutOfStock renderer.js 183
isStopPicked renderer.js 904
isWalkway Warehouse.java 30
itemLoad renderer.js 104
javaCommand route-engine.cjs 18
javaEngine renderer.js 1127
Json Json.java 15
Json Json.java 19
kpi renderer.js 301
layoutBlocks renderer.js 1097
layoutCellCenter renderer.js 1114
Leg RoutePlan.java 24
link EngineTest.java 128
list Warehouse.java 80
loadState renderer.js 156
localDateKey renderer.js 181
locationOptionLabel renderer.js 476
locationOwners renderer.js 88
locationProblem renderer.js 400
main EngineServer.java 20
main EngineTest.java 23
MAX_ORDERS_PER_BATCH renderer.js 70
migrateBatchesToFreeGrid renderer.js 65
migrateInventoryStatuses renderer.js 66
migrateInventoryToFreeGrid renderer.js 64
migrateLocationToFreeGrid renderer.js 63
migrateLocationToTwoSides renderer.js 62
modalRoot renderer.js 3
MODEL_VERSION renderer.js 72
nameTaken renderer.js 398
navigate renderer.js 198
nearestNeighbour RouteOptimizer.java 176
neighbour Warehouse.java 25
next Json.java 140
nextId renderer.js 623
nextOrderNumber renderer.js 721
nextProductId renderer.js 624
Node WarehouseGraph.java 19
node WarehouseGraph.java 76
normalizeGridZone renderer.js 54
normalizeProduct renderer.js 86
normalizeWarehouseLayout renderer.js 56
normalizeZoneToTwoSides renderer.js 52
num EngineTest.java 140
num Warehouse.java 90
openBatches renderer.js 187
openLocationsModal renderer.js 506
openOrderDetails renderer.js 833
openOrderModal renderer.js 731
openProductModal renderer.js 407
openReceiveModal renderer.js 477
openShortPickModal renderer.js 955
openStockHistoryModal renderer.js 547
openWarehouseLayoutModal renderer.js 1285
optimize RouteOptimizer.java 59
optimizeRouteStops renderer.js 1094
ORDER_STAGE_LABEL renderer.js 108
orderActionsMarkup renderer.js 699
orderBlockers renderer.js 656
orderDisplayStatus renderer.js 186
orderHandlingClass renderer.js 98
orderLoad renderer.js 105
orderRow renderer.js 708
orderStage renderer.js 109
packOrder renderer.js 815
pageHeader renderer.js 242
parse Json.java 22
parseCsv renderer.js 566
parseLocation renderer.js 625
parseWholeNumber renderer.js 568
path main.cjs 2
path route-engine.cjs 6
pathNeighbors renderer.js 1033
pathPoints RouteOptimizer.java 279
pathTo ShortestPaths.java 47
peek Json.java 135
pendingHighlight renderer.js 197
performance EngineTest.java 296
pickableStockBySku renderer.js 651
PICKERS renderer.js 71
point RouteOptimizer.java 285
productBySku renderer.js 73
productDisplayStatus renderer.js 184
productRow renderer.js 387
protocol EngineTest.java 312
randomizedExactness EngineTest.java 262
randomRequest EngineTest.java 36

reaches	ShortestPaths.java 44
readArray	Json.java 74
readLiteral	Json.java 125
readNumber	Json.java 114
readObject	Json.java 54
readString	Json.java 89
readValue	Json.java 40
receive	route-engine.cjs 49
recordHistory	renderer.js 171
recordMovement	renderer.js 89
Reference	EngineTest.java 82
Reference	EngineTest.java 88
refreshBehindModal	renderer.js 475
removeOrderFromBatch	renderer.js 982
render	renderer.js 233
renderDashboard	renderer.js 246
renderHistory	renderer.js 1351
renderInventory	renderer.js 364
renderOrders	renderer.js 675
renderPicking	renderer.js 907
request	route-engine.cjs 68
requestRoutePlan	renderer.js 1143
rerenderOrderViews	renderer.js 814
reservedByLocation	renderer.js 96
reservedBySku	renderer.js 95
resetDemoData	renderer.js 1365
routeEngine	main.cjs 11
RouteEngine	route-engine.cjs 8
routeLegsDistance	renderer.js 1206
RouteOptimizer	RouteOptimizer.java 26
RouteOptimizer	RouteOptimizer.java 46
RouteOptimizer	RouteOptimizer.java 49
RoutePlan	RoutePlan.java 20
routePlanKey	renderer.js 1129
samePlace	EngineTest.java 216
SAMPLE_CUSTOMERS	renderer.js 796
saveState	renderer.js 170
searchFrom	RouteOptimizer.java 55
selectedBatchId	renderer.js 154
setupTableFilter	renderer.js 219
shipOrder	renderer.js 816
ShortestPaths	ShortestPaths.java 13
ShortestPaths	ShortestPaths.java 17
shortestPathToAny	renderer.js 1038
size	WarehouseGraph.java 75
skipWhitespace	Json.java 131
sortReadyOrders	renderer.js 673
Square	Warehouse.java 24
square	Warehouse.java 74
start	route-engine.cjs 24
startToCorridorPath	renderer.js 1051
state	renderer.js 151
stepCost	EngineTest.java 221
Stop	Warehouse.java 42
stop	route-engine.cjs 78
stopIsLast	renderer.js 1060
stopKey	renderer.js 1061
STORAGE_KEY	renderer.js 1
str	Warehouse.java 85
syncDashboardPanels	renderer.js 294
syncProduct	renderer.js 77
toast	renderer.js 176
toastElement	renderer.js 4
toggleStopPicked	renderer.js 947
toJson	RoutePlan.java 30
toLocalInputValue	renderer.js 722
Tour	RouteOptimizer.java 30
updateLocationPreview	renderer.js 635
updateReadyBadge	renderer.js 172
upgradeState	renderer.js 111
usableLocation	renderer.js 649
viaJson	EngineTest.java 75
viewFilters	renderer.js 196
Warehouse	Warehouse.java 20
WarehouseGraph	WarehouseGraph.java 16
WarehouseGraph	WarehouseGraph.java 31
warehouseLayoutSvg	renderer.js 1251
warehouseStartingPoint	renderer.js 1037
wireModal	renderer.js 496
write	Json.java 32
writeString	Json.java 191
writeValue	Json.java 156
Zone	Warehouse.java 29
zoneDoorKey	renderer.js 1043
zoneEntryPath	renderer.js 1036